



terv▶▶

practical learning redefined

One point solution for all your placement needs

Linked list Data Structure

A linked list is a linear data structure that includes a series of connected nodes. Here, each node stores the **data** and the **address** of the next node



Linked lists can be of multiple types:

- Singly linked list
- Doubly linked list
- circular linked list

Representation of Linked List

- A data item
- An address of another node

We wrap both the data item and the next node reference

```
static class Node {  
    int value;  
    Node next;  
}
```

Implementation of Linked List

```
public class Dsa {  
  
    static Node head = null;  
  
    static class Node{  
        int data;  
        Node next;  
    }  
}
```

Implementation of Linked List

```
static void add(int d) {  
    // Create newNode  
    Node newNode = new Node();  
    newNode.data = d;  
    newNode.next = null;  
    //Append Node  
    if(head == null) head = newNode;  
    else {  
        Node temp = head;  
        while(temp.next!=null) temp = temp.next;  
        temp.next = newNode;  
    }  
}
```

Implementation of Linked List

```
public static void main(String[] args) {  
    System.out.println(head);  
    add(10);  
    add(30);  
    add(25);  
    System.out.println(head.data);  
    System.out.println(head.next.data);  
    System.out.println(head.next.next.data);  
}
```

Linked List Complexity

Time Complexity:

	Worst case	Average Case
Search	$O(n)$	$O(n)$
Insert	$O(1)$	$O(1)$
Deletion	$O(1)$	$O(1)$

Space Complexity: $O(n)$

Linked List Applications:

- Dynamic memory allocation
- Implemented in stack and queue
- In **undo** functionality of software
- Hash tables, Graphs

Linked List Operations:

- **Traversal** –
access each element of the linked list
- **Insertion** –
adds a new element to the linked list
- **Deletion** –
removes the existing elements
- **Search** –
find a node in the linked list
- **Sort** –
sort the nodes of the linked list

Traverse a Linked List

```
//Traverse
static void display() {
Node temp = head;
while(temp != null) {
System.out.print(temp.data + "->");
temp = temp.next;
}
System.out.println("Null");
}
```

Insert at the beginning

- Allocate memory for new node
- Store data
- Change next of new node to point to head
- Change head to point to recently created node

Insert at the beginning

```
static void ins_f(int d) {  
    // Create newNode  
    Node newNode = new Node();  
    newNode.data = d;  
    newNode.next = null;  
  
    // Insert as head Node  
    newNode.next = head;  
    head = newNode;  
}
```

Insert at the End

- Allocate memory for new node
- Store data
- Traverse to last node
- Change next of last node to recently created node

Insert at the End

```
static void ins_l(int d) {  
    // Create newNode  
    Node newNode = new Node();  
    newNode.data = d;      newNode.next = null;  
  
    //Append Node  
    if(head == null) head = newNode;  
    else {    Node temp = head;  
    while(temp.next!=null)  
        temp = temp.next;  
    temp.next = newNode;  
    }  
}
```

Insert at the Middle

- Allocate memory and store data for new node
- Traverse to node just before the required position of new node
- Change next pointers to include new node in between

Insert at the Middle

```
static void ins_aft(int x, int d) {  
    // Create newNode  
    Node newNode = new Node();  
    newNode.data = d;      newNode.next = null;  
  
    //Traverse to find x  
    Node temp = head;  
    while(temp.data != x) temp = temp.next;  
    //Insert after x  
    newNode.next = temp.next;  
    temp.next = newNode;  
}
```

Insert at the Middle

```
static void ins_bf(int x,int d) {  
    // Create newNode  
    Node newNode = new Node(d);  
  
    //Traverse to find x  
    Node prev = null, temp = head;  
    while(temp.data != x) {  
        prev = temp; temp = temp.next;  
    }  
    newNode.next = prev.next;  
    prev.next = newNode;  
}
```

Delete from a Linked List

You can delete either from the beginning, end or from a particular position.

Delete from beginning

- Point head to the second node

Delete from beginning

```
static void del_f() {  
  
    //Check head node  
    if(head == null) return;  
  
    //Point head to next of current head  
    head = head.next;  
}
```

Delete from end

- Traverse to second last element
- Change its next pointer to null

Delete from end

```
static void del_l() {  
    //Check second last Node  
    Node prev = null, temp = head;  
  
    while(temp.next != null) {  
        prev = temp;  
        temp = temp.next;  
    }  
  
    prev.next = null;  
}
```

Delete from middle

- Traverse to element before the element to be deleted
- Change next pointers to exclude the node from the chain

Delete from middle

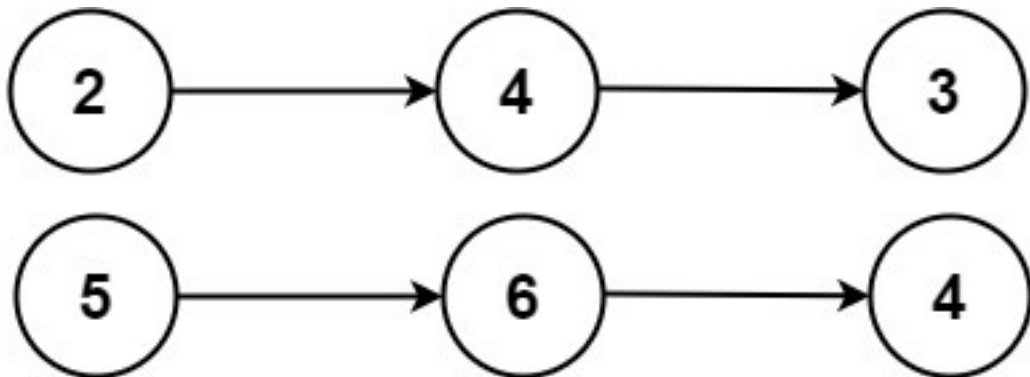
```
static void del(int x) {  
    //Traverse to X  
    Node prev = null, temp = head;  
  
    while(temp.data != x) {  
        prev = temp;  
        temp = temp.next;  
    }  
    prev.next = temp.next;  
}
```

Challenge :

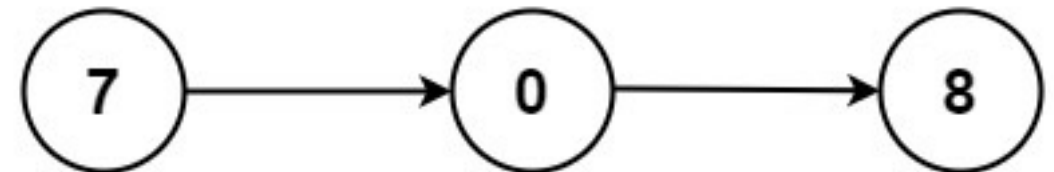
You are given two **non-empty** linked lists representing two non-negative integers. The digits are stored in **reverse order**, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Sample IP:



Sample OP:



Search an Element on a Linked List

You can search an element on a linked list using a loop using the following steps. We are finding item on a linked list.

- Make head as the current node.
- Run a loop until the current node is NULL because the last element points to NULL.
- In each iteration, check if the key of the node is equal to item. If it the key matches the item, return true otherwise return false.

Search an Element on a Linked List

```
boolean search(int x) {  
    Node temp = head;  
    while(temp != null && temp.data != x) {  
        temp = temp.next;  
    }  
    return temp == null ? false : true;  
}
```

Sort Elements of a Linked List

(Bubble Sort)

- Make the head as the current node and create another node index for later use.
- If head is null, return.
- Else, run a loop till the last node (i.e. NULL).
- In each iteration, follow the following step 5-6.
- Store the next node of current in index.
- Check if the data of the current node is greater than the next node. If it is greater, swap current and index.

Sort Elements of a Linked List

```
void bubble_sort() {  
Node current = head, index = null;  
if(head == null) return;  
while(current != null) { index = current.next;  
while(index != null) {  
if(current.data > index.data) {  
int t = current.data;  
current.data = index.data;      index.data = t;  
}index = index.next;  
}current = current.next;  
}  
}
```



Doubly Linked List

We add a pointer to the previous node in a doubly-linked list. Thus, we can go in either direction: forward or backward.

Doubly Linked List (Initialisation)

```
class List{  
Node head = null;
```

```
class Node{  
Node prev;    int data;    Node next;
```

```
Node(int d){  
prev = null; data = d;    next = null;  
}  
  
}
```

Doubly Linked List (Append)

```
void add(int d) {  
    Node newNode = new Node(d);  
    //append node  
    if(head == null) head = newNode;  
    else {  
        Node temp = head;  
        while(temp.next != null)  
            temp = temp.next;  
        newNode.prev = temp; temp.next = newNode;  
    }  
}
```

Doubly Linked List

//Traverse from head to tail

```
void display_h_t() {  
    if(head == null) return;  
    Node temp = head;  
  
    while(temp != null) {  
        System.out.print(temp.data+"->");  
        temp = temp.next;  
    }  
    System.out.println("Null");  
}
```

Doubly Linked List

```
//Traverse from tail to head
void display_t_h() {
    if(head == null) return;
    Node temp = head;
    //Traverse to tail
    while(temp.next != null) temp = temp.next;
    //Traverse to head
    while(temp != null) {
        System.out.print(temp.data+"->");
        temp = temp.prev;
    }System.out.println("Null");
}
```

Doubly Linked List

```
public class Dsa {  
    public static void main(String[] args) {  
        List l = new List();  
        l.add(8); l.add(4);  
        l.add(6); l.add(12);  
        l.add(10);  
        l.display_h_t();  
        l.display_t_h();  
    }  
}
```

OP:

8->4->6->12->10->Null

10->12->6->4->8->Null

Circular Linked List



A circular linked list is a variation of a linked list in which the last element is linked to the first element. This forms a circular loop.

Circular Linked List

A circular linked list can be either singly linked or doubly linked.

- for singly linked list, next pointer of last item points to the first item
- In the doubly linked list, prev pointer of the first item points to the last item as well.

Doubly Circular Linked List (Append)

```
void add(int d) {  
    Node newNode = new Node(d);  
    if(head == null) {  
        head = newNode; newNode.prev = head;  
        newNode.next = head;  
    }  
    else {  
        Node temp = head;  
        while(temp.next != head) temp = temp.next;  
        newNode.prev = temp; temp.next = newNode;  
        newNode.next = head; head.prev = newNode;  
    }  
}
```

Singly Circular Linked List (Append)

```
void add(int d) {  
    Node newNode = new Node(d);  
    if(head == null) {  
        head = newNode;    newNode.next = head;  
    }  
    else {  
        Node temp = head;  
        while(temp.next != head)    temp = temp.next;  
  
        temp.next = newNode;  
        newNode.next = head;  
    }  
}
```

Singly Circular Linked List (Traverse)

```
void display() {  
    if(head == null) return;
```

```
    Node temp = head;
```

```
    do{  
        System.out.print(temp.data+"->");  
        temp = temp.next;  
    }while(temp != head);
```

```
    System.out.println("Null");  
}
```

Doubly Circular Linked List (Traversal)

```
// Traverse from head to tail
void display_h_t() {
    if(head == null) return;
    Node temp = head;
    do{
        System.out.print(temp.data+"->");
        temp = temp.next;
    }while(temp != head);
    System.out.println("Null");
}
```

Doubly Circular Linked List (Traversal)

```
// Traverse from tail to head
void display_t_h() {
    if(head == null) return;
    Node temp = head.prev;
    do {
        System.out.print(temp.data+"->");
        temp = temp.prev;
    } while(temp != head.prev);
    System.out.println("Null");
}
```

Stack Using Linked List

```
class Stack{
```

```
Node top = null;
```

```
class Node{
```

```
int data;
```

```
Node next;
```

```
Node(int d){
```

```
data = d; next = null;
```

```
}
```

```
}
```

Stack Using Linked List

```
void push(int d) {  
    Node newNode = new Node(d);  
    if(isFull(newNode)) {  
        System.out.println("Heap Overflow");  
        return;  
    }  
    newNode.next = top;  
    top = newNode;  
}  
  
boolean isFull(Node newNode){  
    return newNode == null;  
}
```

Stack Using Linked List

```
int pop() {  
    if(isEmpty()) {  
        System.out.println("Stack UnderFlow");  
        return -1;  
    }  
    int res = top.data;  
    top = top.next;  
    return res;  
}  
boolean isEmpty() {  
    return top == null;  
}
```

Stack Using Linked List

```
int peek() {  
    if(isEmpty()) {  
        System.out.println("Stack is Empty");  
    }  
    return top.data;  
}  
  
void display() {  
    if (top == null) return;  
    Node temp = top;  
    while(temp != null) {  
        System.out.print(temp.data+" ");  
        temp = temp.next;  
    }  
}
```

Stack Using Linked List

```
public class Dsa {  
    static Scanner sc = new Scanner(System.in);  
    public static void main(String[] args) {  
        Stack s = new Stack();  
        s.push(10); s.push(40);  
        s.push(80);  
        s.push(20);  
        System.out.println(s.peek());  
        s.pop();  
        s.display();  
    }  
}
```

OP:

20

80 40 10

Queue Using Linked List

```
package sample;
import java.util.*;
//onecompiler.com/java/3zctkym7r
class Queue{
Node front = null, rear = null;
class Node{
int data;
Node next;
Node(int d){
data = d; next = null;
}
}
```

Queue Using Linked List

```
void enQ(int d) {  
    Node newNode = new Node(d);  
    if(newNode == null)//isFull  
    {  
        System.out.println("Queue is Full");  
        return;  
    }  
    if(front == null) front = rear = newNode;  
    else {  
        rear.next = newNode; rear = newNode;  
    }  
}
```

Queue Using Linked List

```
int deQ() {  
    if(front == null) { //isEmpty  
        System.out.println("Queue is empty");  
        return -1;  
    }
```

```
    int res = front.data;  
    front = front.next;  
    return res;  
}
```

Queue Using Linked List

```
int peek() {  
    if(front == null) { //isEmpty  
        System.out.println("Queue is empty");  
        return -1;  
    }  
    return front.data;  
}
```

Queue Using Linked List

```
void display() {  
    if(front == null) { //isEmpty  
        System.out.println("Queue is empty");  
        return;  
    }  
    Node temp = front;  
    while(temp != null) {  
        System.out.print(temp.data+" ");  
        temp = temp.next;  
    }  
}  
}
```

Queue Using Linked List

```
public class Dsa {  
    static Scanner sc = new Scanner(System.in);  
    public static void main(String[] args) {  
        Queue q = new Queue();  
        q.enQ(10); q.enQ(50);  
        q.enQ(90); q.enQ(76);  
        q.enQ(54); q.enQ(4);  
        q.deQ();  
        q.display();  
    }  
}
```

OP:

50 90 76 54 4

Tree Data Structure

Other data structures such as **arrays**, **linked list**, **stack**, and **queue** are **linear data structures** that store data sequentially. In order to perform any operation in a linear data structure, the **time complexity increases** with the increase in the data size. But, it is not acceptable in today's computational world.

Tree Terminologies

1. Node
2. Edge
3. Root
4. Height of a Node
5. Depth of a Node
6. Height of a Tree
7. Degree of a Node
8. Forest

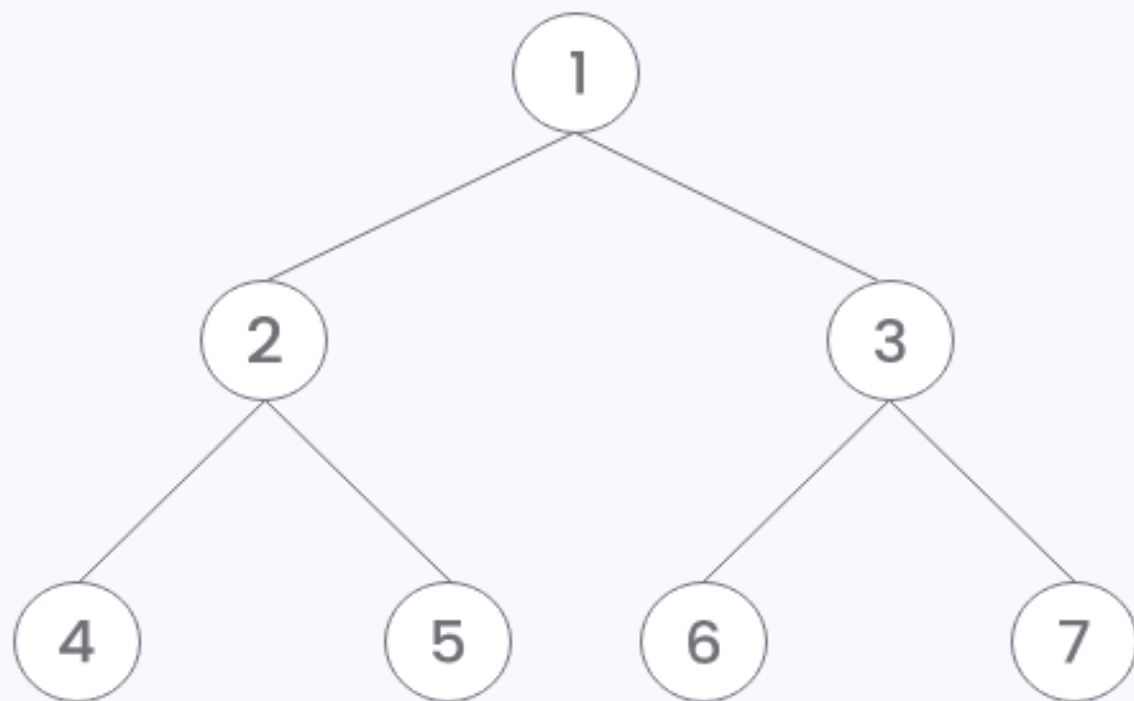
Types of Tree

1. Binary Tree
2. Binary Search Tree
3. AVL Tree
4. B-Tree

Tree Traversal

1. Inorder traversal (Left – Root – Right)
2. Preorder traversal (Root – Left – Right)
3. Postorder traversal (Left – Right – Root)

Tree Traversal Techniques



Inorder Traversal

4	2	5	1	6	3	7
---	---	---	---	---	---	---

Preorder Traversal

1	2	4	5	3	6	7
---	---	---	---	---	---	---

Postorder Traversal

4	5	2	6	7	3	1
---	---	---	---	---	---	---

Example for Traversal:

```
static void inorder(Node root) {  
    if (root == null)  
        return;  
    // Traverse left  
    inorder(root.left);  
    // Traverse root  
    System.out.print(root.item + "->");  
    // Traverse right  
    inorder(root.right);  
}
```

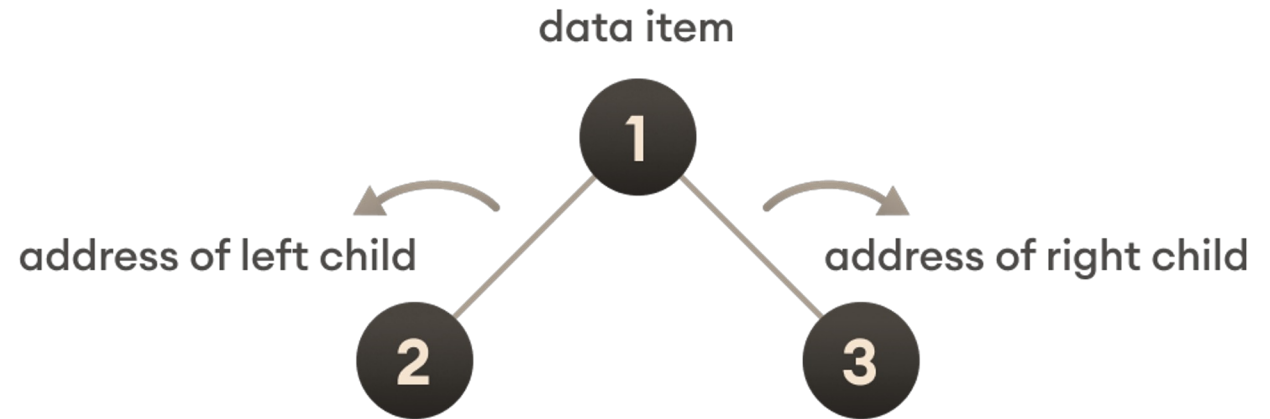
Example for Traversal:

```
static void preorder(Node root) {  
    if (root == null)  
        return;  
    // Traverse root  
    System.out.print(root.item + "->");  
    // Traverse left  
    preorder(root.left);  
    // Traverse right  
    preorder(root.right);  
}
```

Example for Traversal:

```
static void postorder(Node root) {  
    if (root == null)  
        return;  
    // Traverse left  
    postorder(root.left);  
    // Traverse right  
    postorder(root.right);  
    // Traverse root  
    System.out.print(root.item + "->");  
}
```

Binary Tree



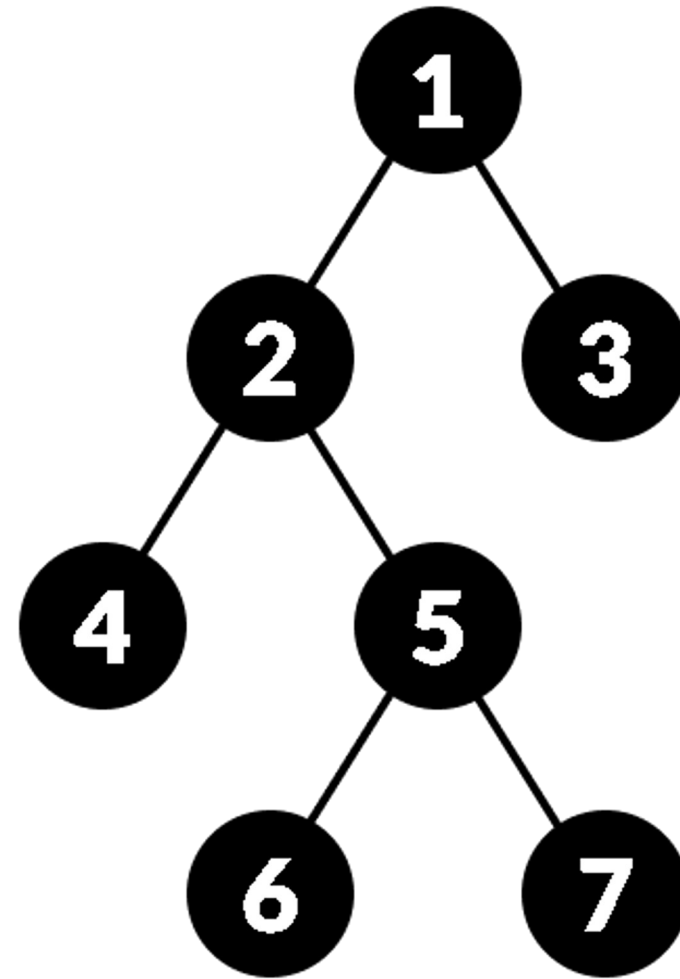
- data item
- address of left child
- address of right child

Types of Binary Tree

1. Full Binary Tree
2. Perfect Binary Tree
3. Complete Binary Tree
4. Degenerate or Pathological Tree
5. Skewed Binary Tree
6. Balanced Binary Tree

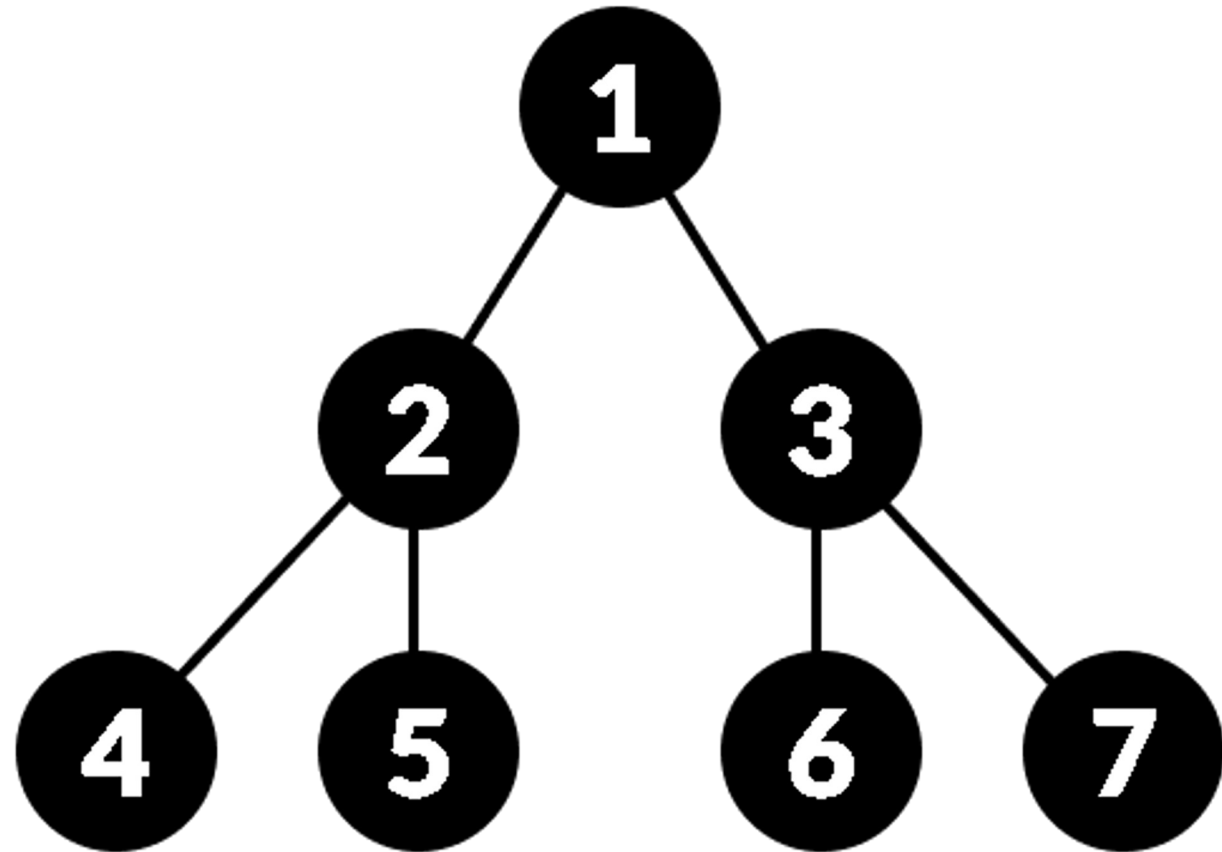
Full Binary Tree

A full Binary tree is a special type of binary tree in which **every parent node/internal node** has either **two** or **no children**.



Perfect Binary Tree

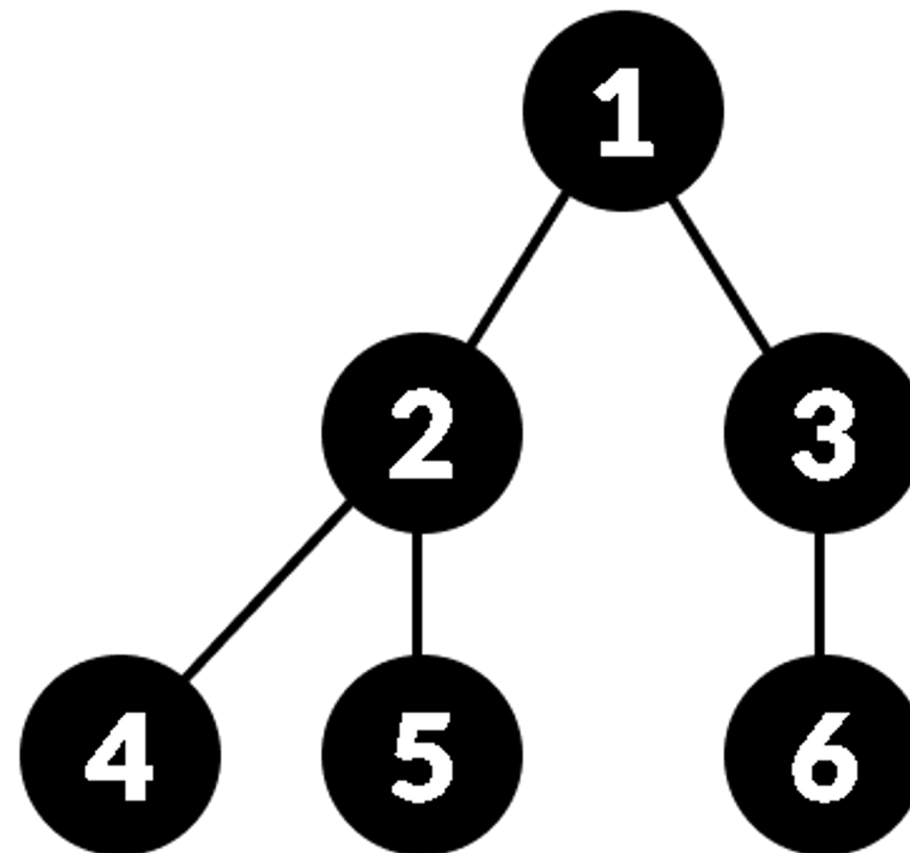
A perfect binary tree is a type of binary tree in which **every internal node** has **exactly two child nodes** and all the **leaf nodes** are at the **same level**.



Complete Binary Tree

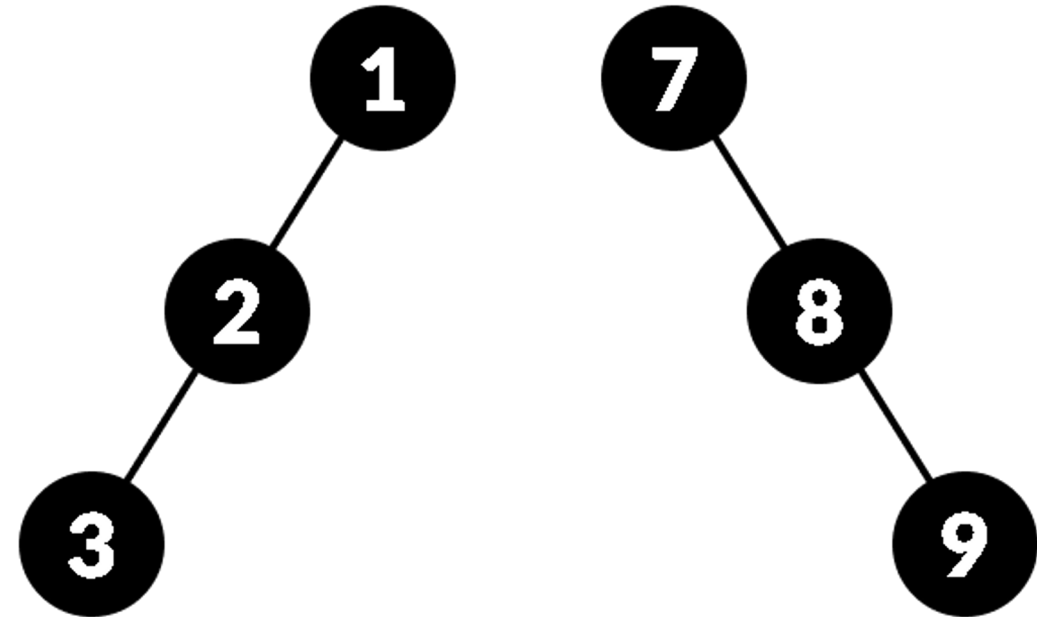
A complete binary tree is just like a full binary tree, but with two major differences

- Every level must be completely filled
- All the leaf elements must lean towards the left.
- The last leaf element might not have a right sibling i.e. a complete binary tree doesn't have to be a full binary tree.



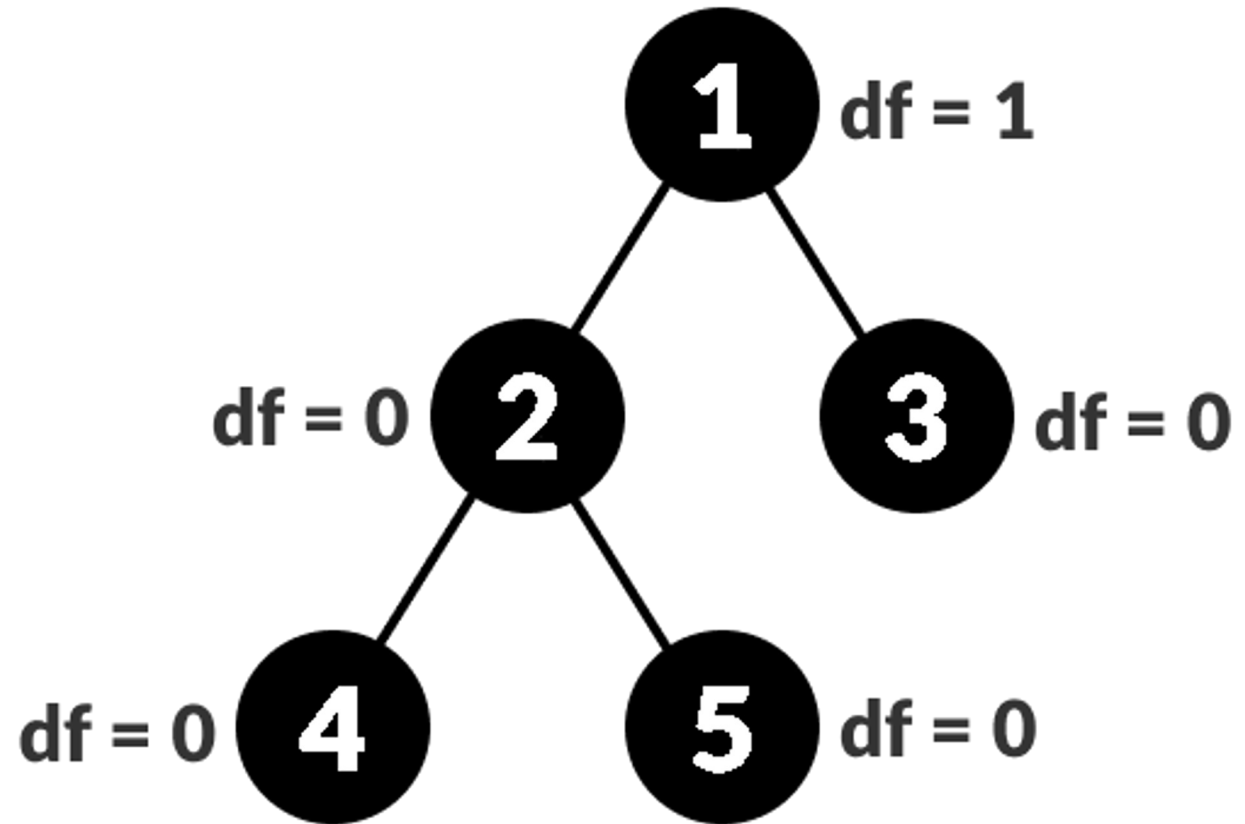
Skewed Binary Tree

A skewed binary tree is a pathological/degenerate tree in which the tree is either dominated by the left nodes or the right nodes. Thus, there are two types of skewed binary tree: **left-skewed binary tree** and **right-skewed binary tree**.



Balanced Binary Tree

It is a type of binary tree in which the difference between the **height of the left and the right subtree for each node** is either 0 or 1.



Binary Tree Representation

```
class Node {  
    int data;  
    Node left, right;  
  
    Node(int d) {  
        data = d;  
        left = right = null;  
    }  
}
```

Binary Tree Representation

```
class BinaryTree {  
Node root;  
BinaryTree(int d) {  
root = new Node(d);  
}
```

```
BinaryTree() {  
root = null;  
}  
//inorder, preorder, postorder  
}
```

Binary Tree Representation

```
public static void main(String[] args) {  
    BinaryTree tree = new BinaryTree();  
    tree.root = new Node(1);  
    tree.root.left = new Node(2);  
    tree.root.right = new Node(3);  
    tree.root.left.left = new Node(4);  
  
    tree.InOrder(tree.root);  
    tree.PreOrder(tree.root);  
    tree.PostOrder(tree.root);  
}
```

Full Binary Tree Theorems

Let, i = the number of internal nodes
 n = be the total number of nodes
 l = number of leaves
 λ = number of levels

1. The number of leaves is $i + 1$.
2. The total number of nodes is $2i + 1$.
3. The number of internal nodes is $(n - 1) / 2$.
4. The number of leaves is $(n + 1) / 2$.
5. The total number of nodes is $2l - 1$.
6. The number of internal nodes is $l - 1$.
7. The number of leaves is at most $2^{\lambda - 1}$.

Full Binary Tree

```
// Check for Full Binary Tree
boolean isFullBinaryTree(Node root) {
// Checking tree emptiness
if (root == null)
return true;
// Checking the children
if (root.left == null && root.right == null)
return true;
if ((root.left != null) && (root.right != null))
return (isFullBinaryTree(root.left) &&
isFullBinaryTree(root.right));
return false;
}
```

Full Binary Tree

Perfect Binary Tree

Recursively, a perfect binary tree can be defined as:

1. If a single node has no children, it is a perfect binary tree of height $h = 0$,
2. If a node has $h > 0$, it is a perfect binary tree if both of its subtrees are of height $h - 1$ and are non-overlapping.

Perfect Binary Tree -> depth

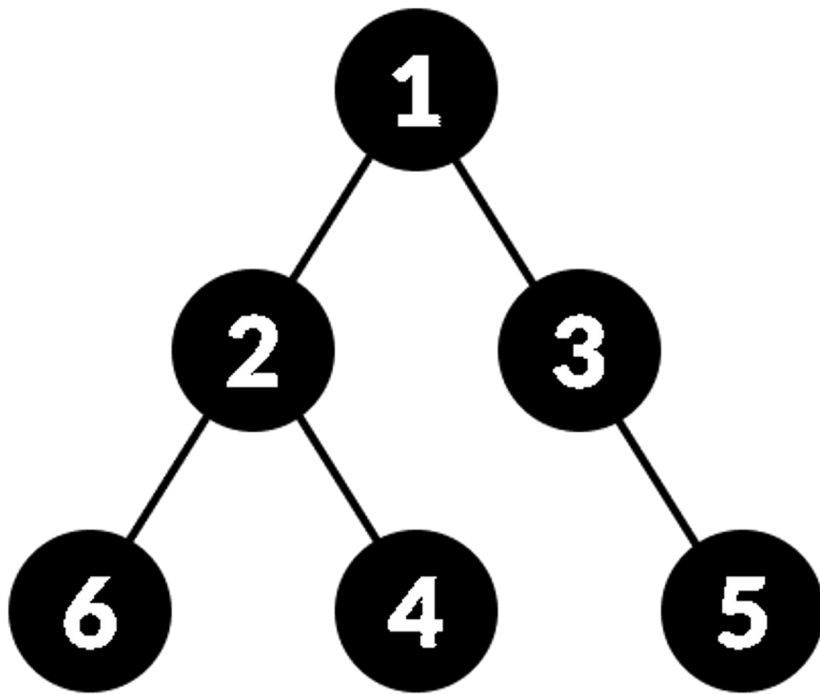
```
int depth(Node root) {  
    int d = 0;  
    while (root != null) {  
        d++;  
        root = root.left;  
    }  
    return d;  
}
```

Perfect Binary Tree -> using depth

```
boolean is_perfect(Node root, int d, int level) {  
    // Check if the tree is empty  
    if (root == null)  
        return true;  
    // If for children  
    if (root.left == null && root.right == null)  
        return (d == level + 1);  
    if (root.left == null || root.right == null)  
        return false;  
    return is_perfect(root.left, d, level + 1) &&  
        is_perfect(root.right, d, level + 1);  
}
```

Perfect Binary Tree -> Wrapping Class

```
boolean isPerfect(Node root) {  
    int d = depth(root);  
    return is_perfect(root, d, 0);  
}
```



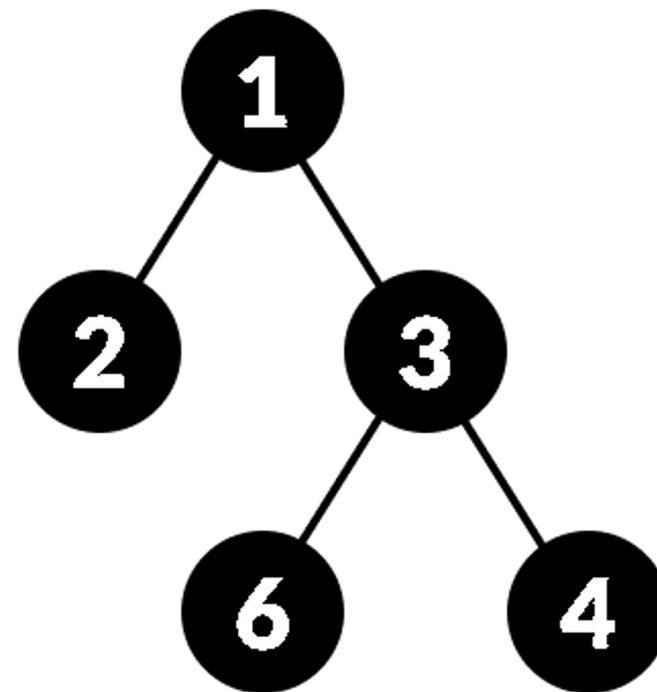
✗ Full Binary Tree

✗ Complete Binary Tree

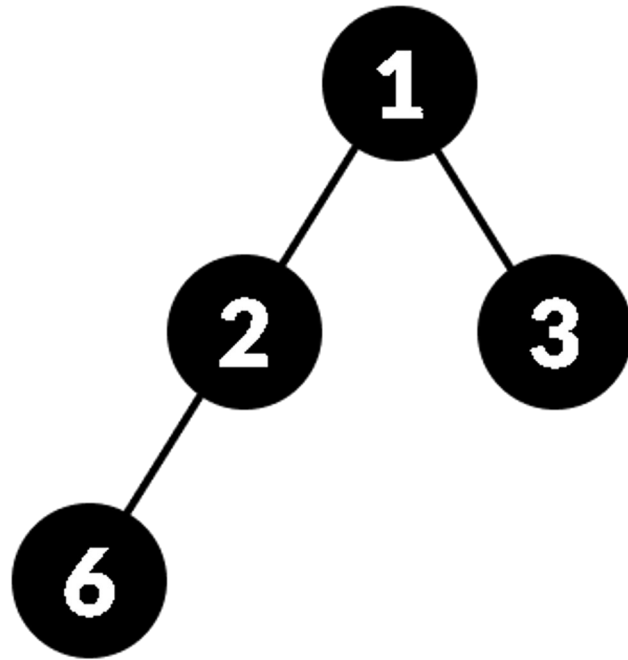
Full Binary Tree vs Complete Binary Tree



Full Binary Tree vs Complete Binary Tree



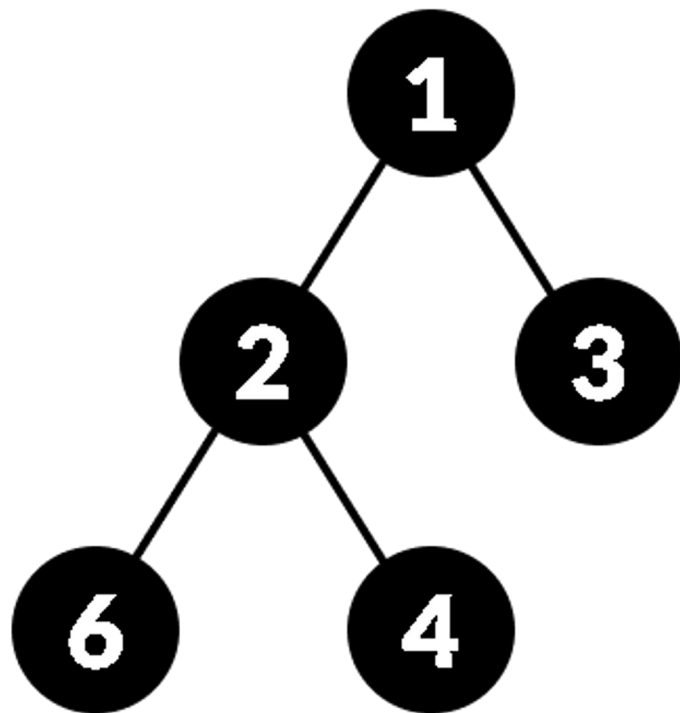
- ✓ Full Binary Tree
- ✗ Complete Binary Tree



✗ Full Binary Tree

✓ Complete Binary Tree

**Full
Binary
Tree vs
Complete
Binary
Tree**



- ✓ Full Binary Tree
- ✓ Complete Binary Tree

Full Binary Tree vs Complete Binary Tree

How a Complete Binary Tree is Created?

Select the first element of the list to be the root node.
(no. of elements on level- l : 1)



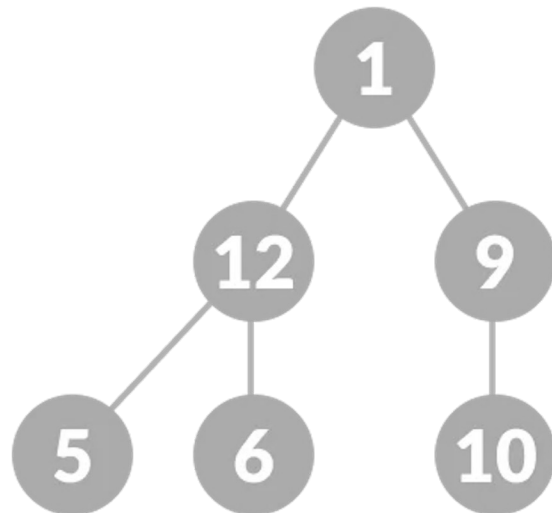
How a Complete Binary Tree is Created?

Put the second element as a left child of the root node and the third element as the right child. (no. of elements on level-II: 2)



How a Complete Binary Tree is Created?

- Put the next two elements as children of the left node of the second level. Again, put the next one element as left children of the right node of the second level (no. of elements on level-III: 3 elements).
- Keep repeating until you reach the last element.



Count No. Of Nodes

```
int count_N_Nodes(Node root) {  
    if (root == null)  
        return (0);  
    return (1 +  
            count_N_Nodes(root.left) +  
            count_N_Nodes(root.right));  
}
```

Check Complete Binary Tree

```
boolean checkComplete(Node root, int index, int
numberNodes) {
// Check if the tree is empty
if (root == null)
return true;
if (index >= numberNodes)
return false;
return (checkComplete(root.left, 2 * index + 1,
numberNodes)
&& checkComplete(root.right, 2 * index + 2,
numberNodes));
}
```

Complete Binary Tree -> Wrapping count_N_Nodes and checkComplete

```
boolean isComplete(Node root) {  
    int N = count_N_Nodes(root);  
    return checkComplete(root, 0, N);  
}
```

Relationship between array indexes and tree element

If the index of any element in the array is i ,

- Element in $2i+1$ index will become the left child
- Element in $2i+2$ index will become the right child.

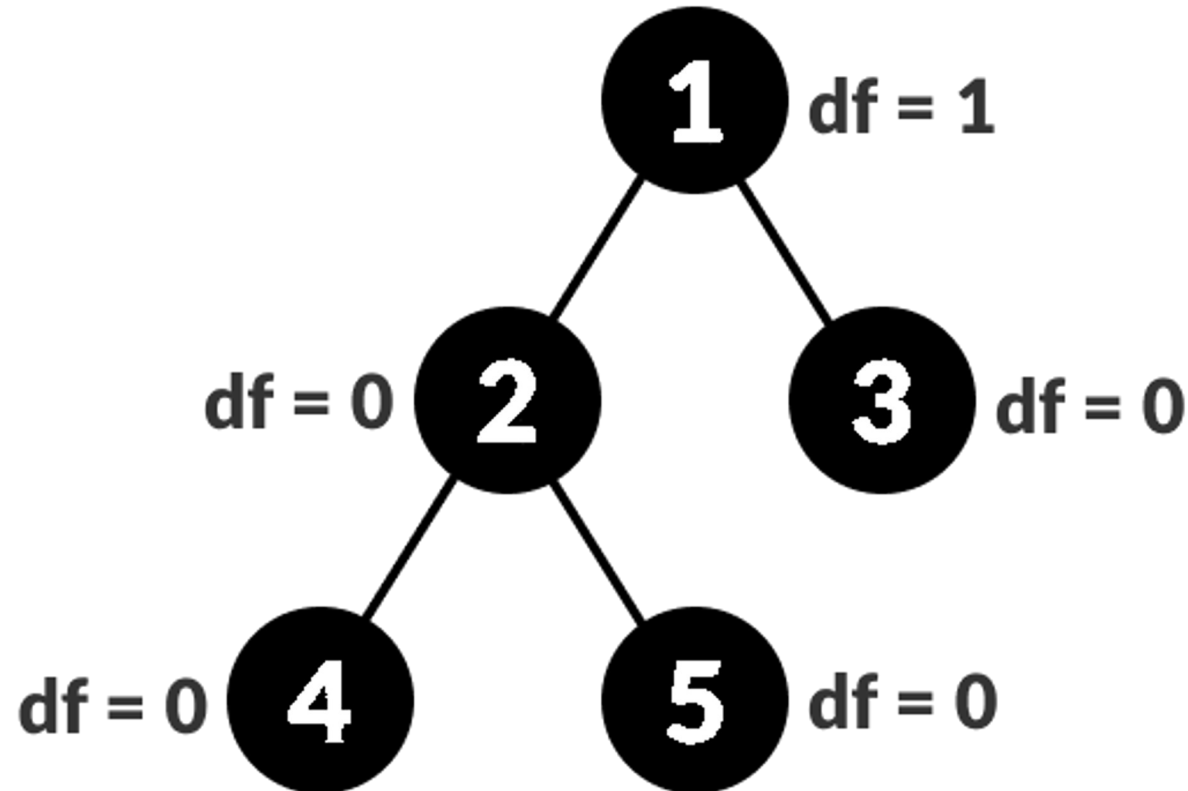
Also, the parent of any element at index i is given by the lower bound of $(i-1)/2$

Balanced Binary Tree

The conditions for a height-balanced binary tree:

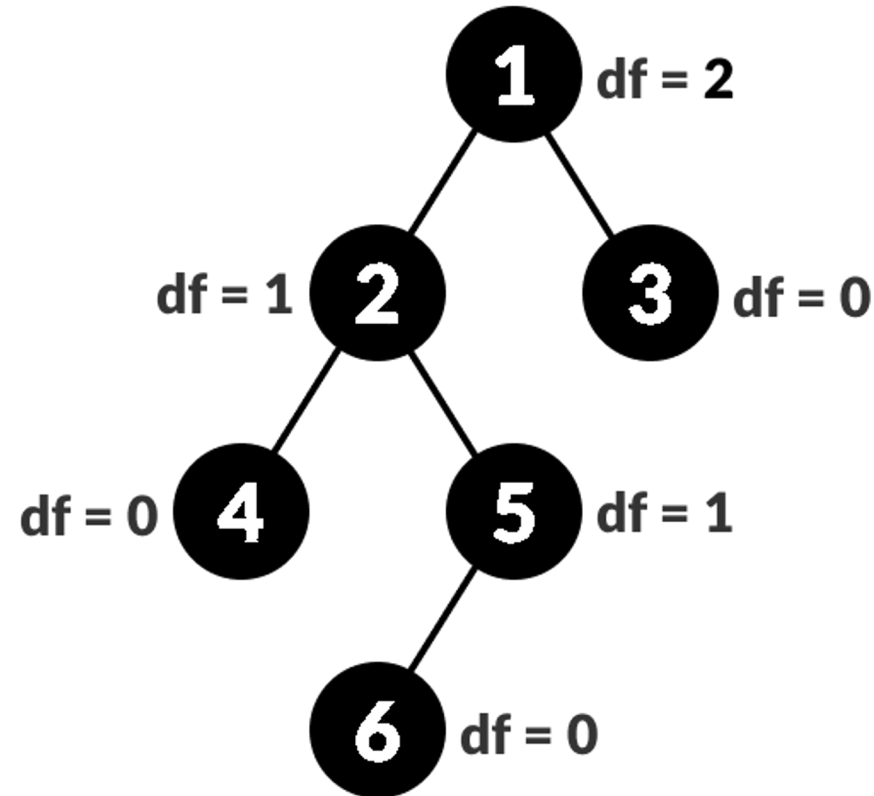
- Difference between the left and the right subtree for any node is not more than one
- The left subtree is balanced
- The right subtree is balanced

Balanced Binary Tree



Balanced Binary Tree with depth at each level

Balanced Binary Tree



$df = |\text{height of left child} - \text{height of right child}|$

Unbalanced Binary Tree with depth at each level

Balanced Binary Tree

```
class BinaryTree {  
Node root = null;  
class Height {  
int height = 0;  
}  
  
boolean checkHeightBalance(Node root, Height height)  
{  
// Check for emptiness  
if (root == null) {  
height.height = 0;  
return true;  
}
```

Balanced Binary Tree

```
Height leftHeight_a = new Height(),  
rightHeight_a = new Height();
```

```
boolean l = checkHeightBalance(root.left,  
leftHeight_a);
```

```
boolean r = checkHeightBalance(root.right,  
rightHeight_a);
```

Balanced Binary Tree

```
int leftHeight = leftHeight_a.height,  
    rightHeight = rightHeight_a.height;  
height.height = (leftHeight > rightHeight ?  
    leftHeight : rightHeight) + 1;  
if ((leftHeight - rightHeight >= 2) ||  
    (rightHeight - leftHeight >= 2))  
    return false;  
else  
    return l && r;  
}
```

Binary Search Tree(BST)

Binary search tree is a data structure that quickly allows us to maintain a sorted list of numbers.

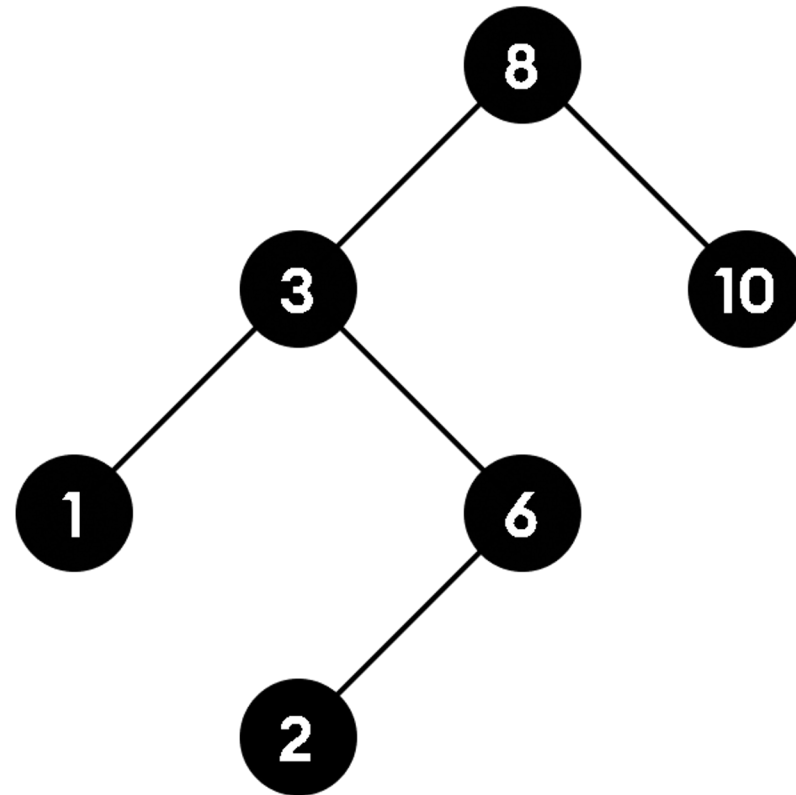
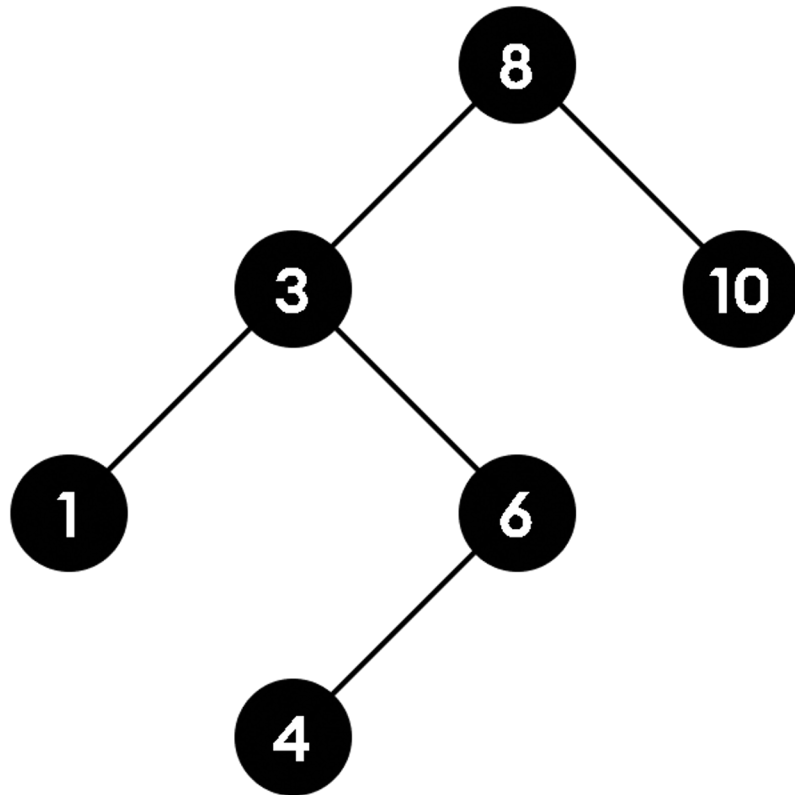
- It is called a binary tree because each tree node has a maximum of two children.
- It is called a search tree because it can be used to search for the presence of a number in $O(\log(n))$ time.

Binary Search Tree(BST)

The properties that separate a binary search tree from a regular **binary tree** is

- All nodes of left subtree are less than the root node
- All nodes of right subtree are more than the root node
- Both subtrees of each node are also BSTs i.e. they have the above two properties

Binary Search Tree(BST)



Search Operation

Algorithm:

If `root == NULL`

return `NULL`;

If `number == root->data`

return `root->data`;

If `number < root->data`

return `search(root->left)`

If `number > root->data`

return `search(root->right)`

Insert Operation

Algorithm:

if node == NULL

return createNode(data)

if (data < node.data)

node.left = insert(node.left, data);

else if (data > node.data)

node.right = insert(node.right, data);

return node;

Insert Operation

```
class BinarySearchTree {  
Node root = null;  
void insert(int data) {  
    root = insertNode(root, data);  
}  
// Insert Data in the tree  
Node insertNode(Node root, int data) {  
    // Return a new node if the tree is empty  
    if (root == null) {  
        root = new Node(data);  
        return root;  
    }  
}
```

Insert Operation

```
// Traverse to the correct place and insert  
the node
```

```
if (data < root.data)  
    root.left = insertNode(root.left, data);  
else if (data > root.data)  
    root.right = insertNode(root.right, data);  
return root;  
}  
}
```

Deletion Operation

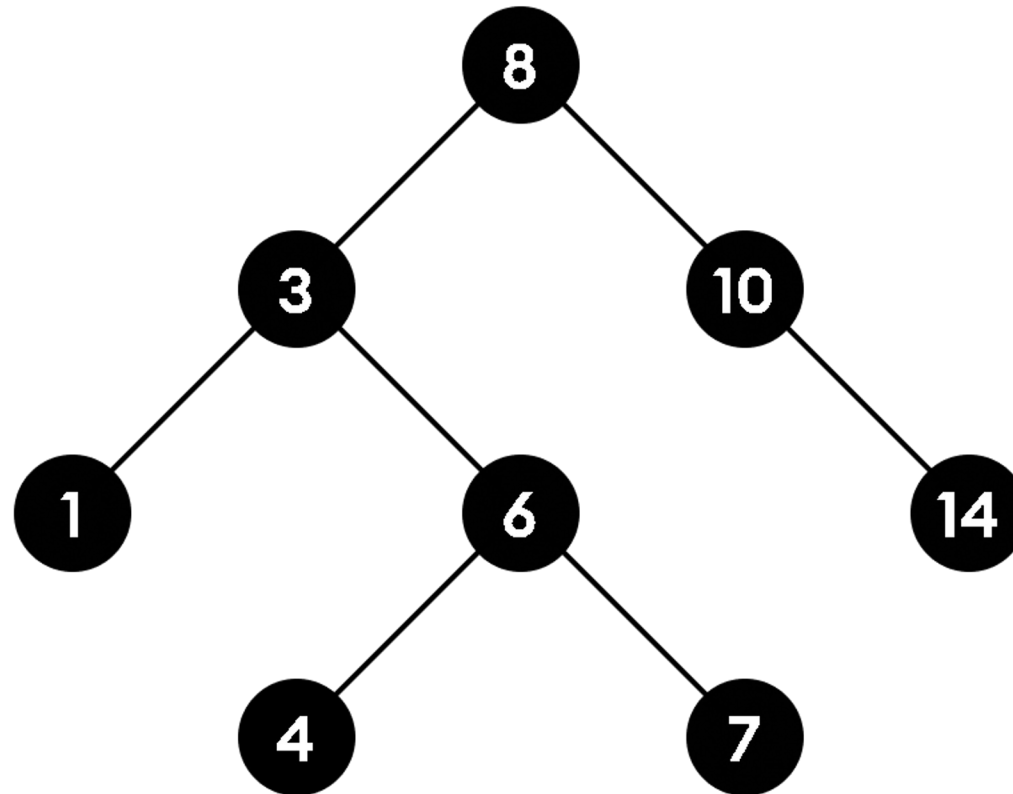
There are three cases for deleting a node from a binary search tree.

Case I:

In the first case, the node to be deleted is the leaf node.

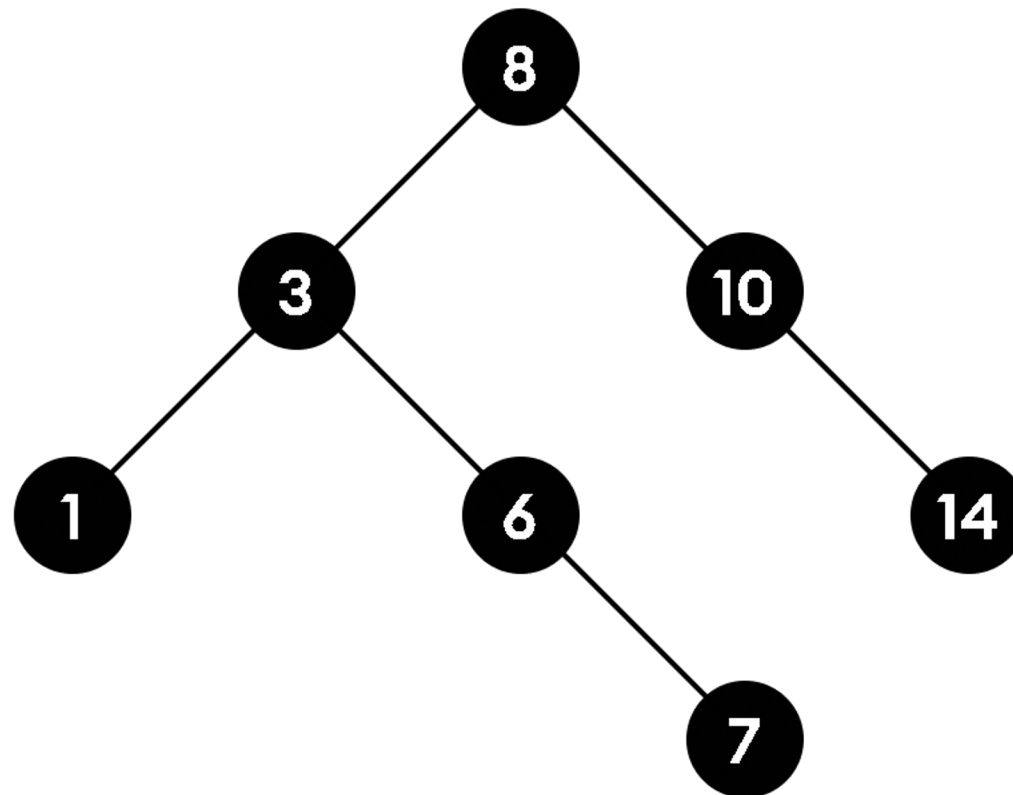
In such a case, simply delete the node from the tree.

Deletion Operation



4 is to be deleted

Deletion Operation



Delete the Node

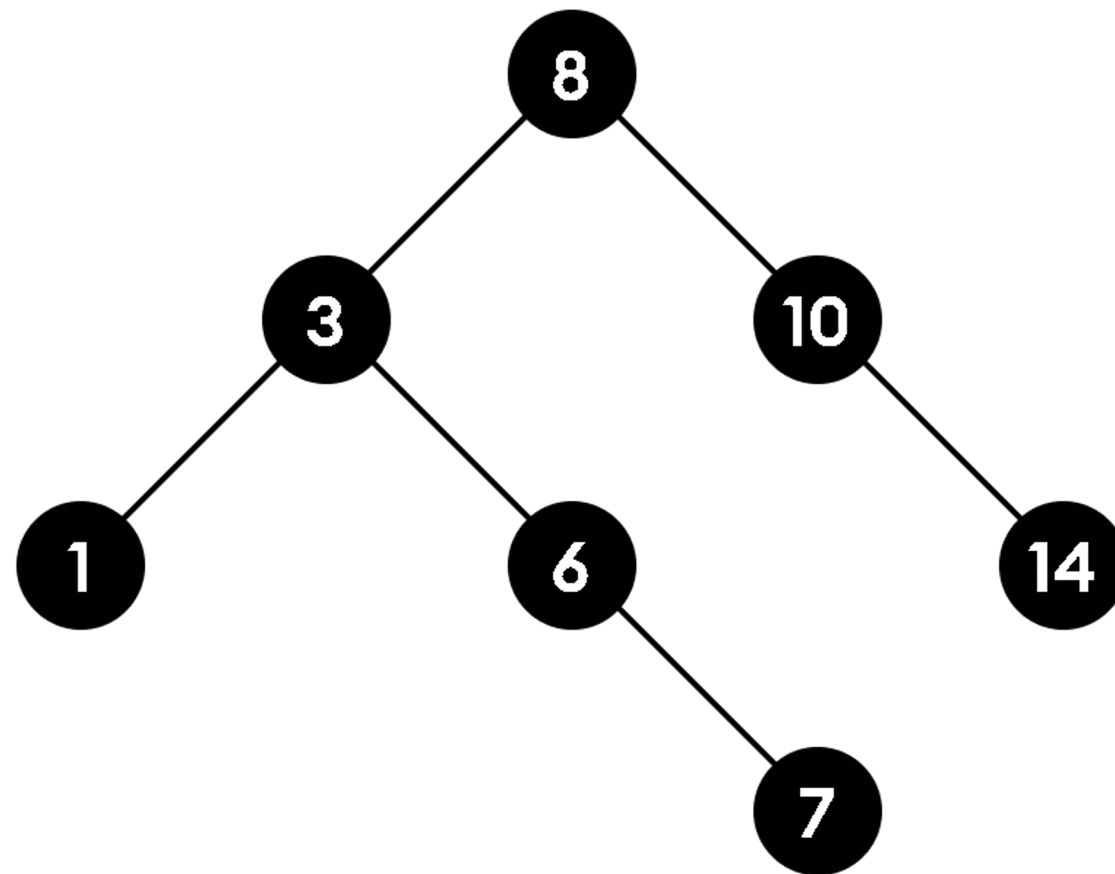
Deletion Operation

Case II :

In the second case, the node to be deleted lies has a single child node. In such a case follow the steps below:

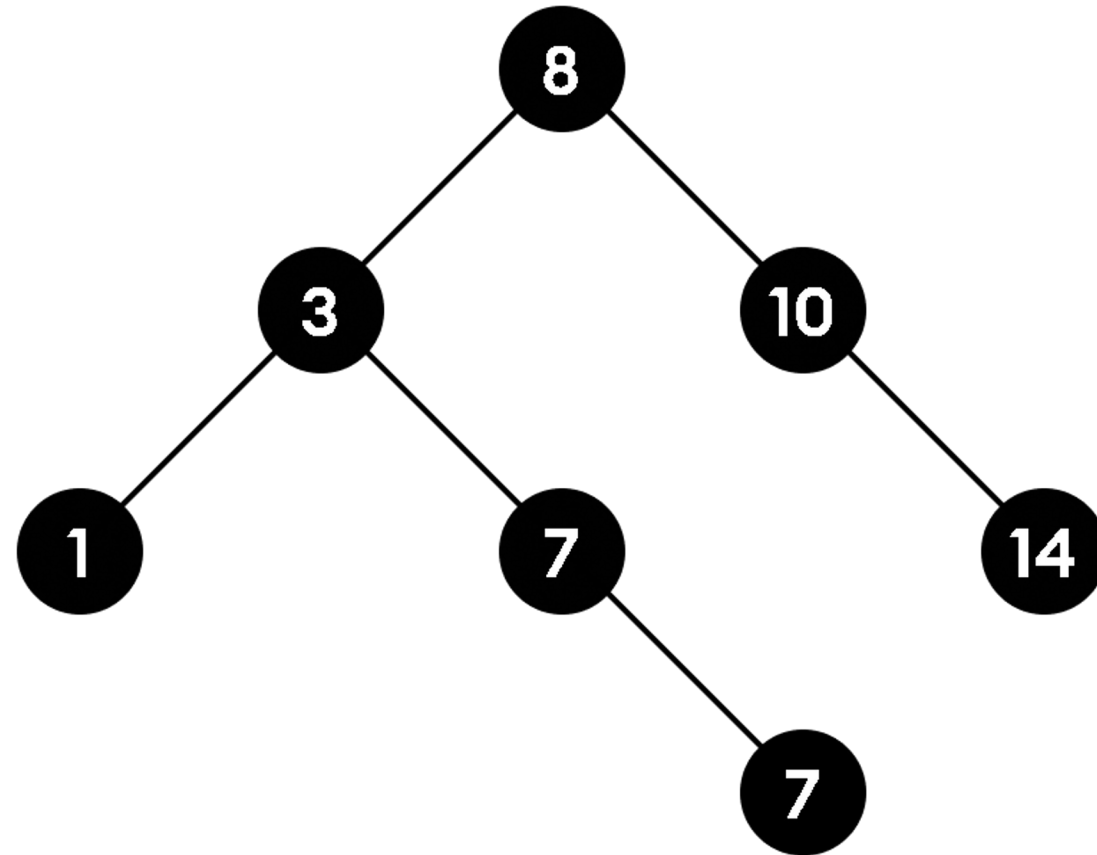
- Replace that node with its child node.
- Remove the child node from its original position.

Deletion Operation



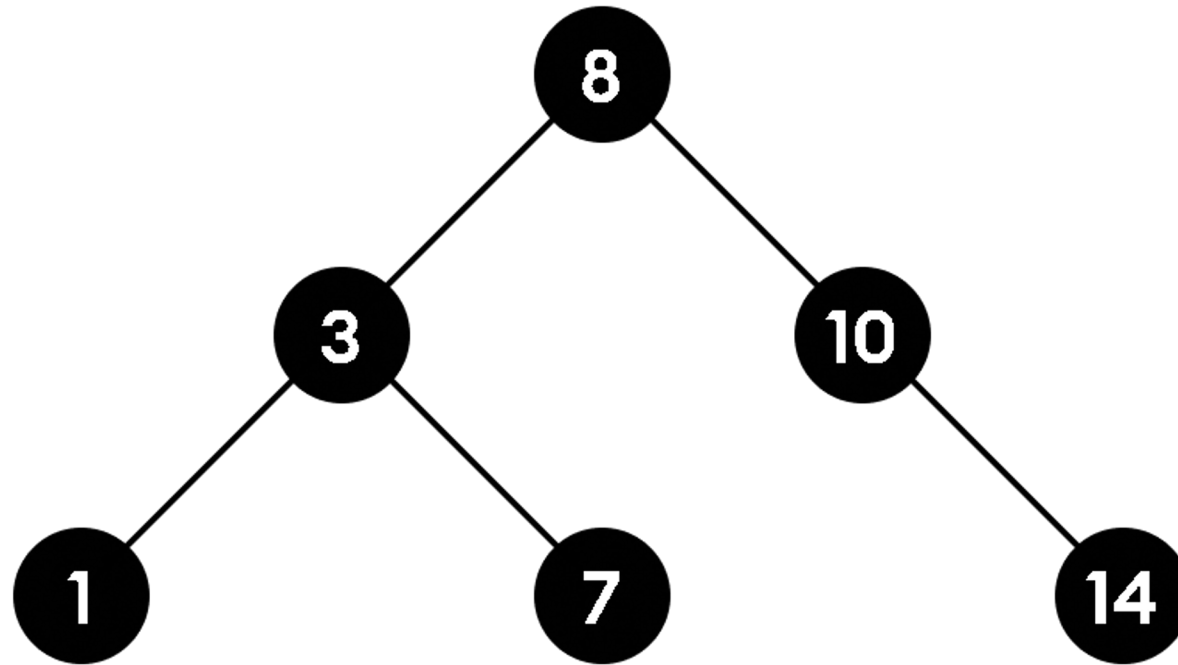
6 is to be deleted

Deletion Operation



copy the value of its child to the node and delete the child

Deletion Operation



Final Tree

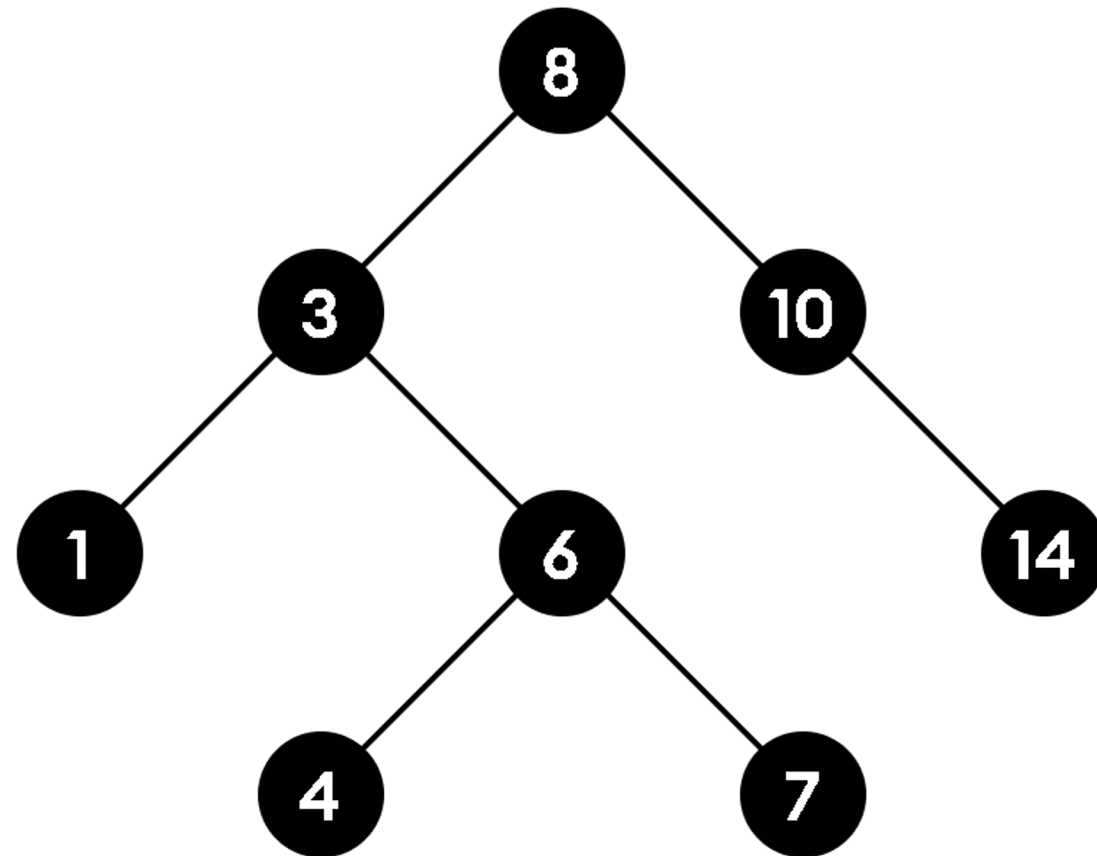
Deletion Operation

Case III :

In the third case, the node to be deleted has two children. In such a case follow the steps below:

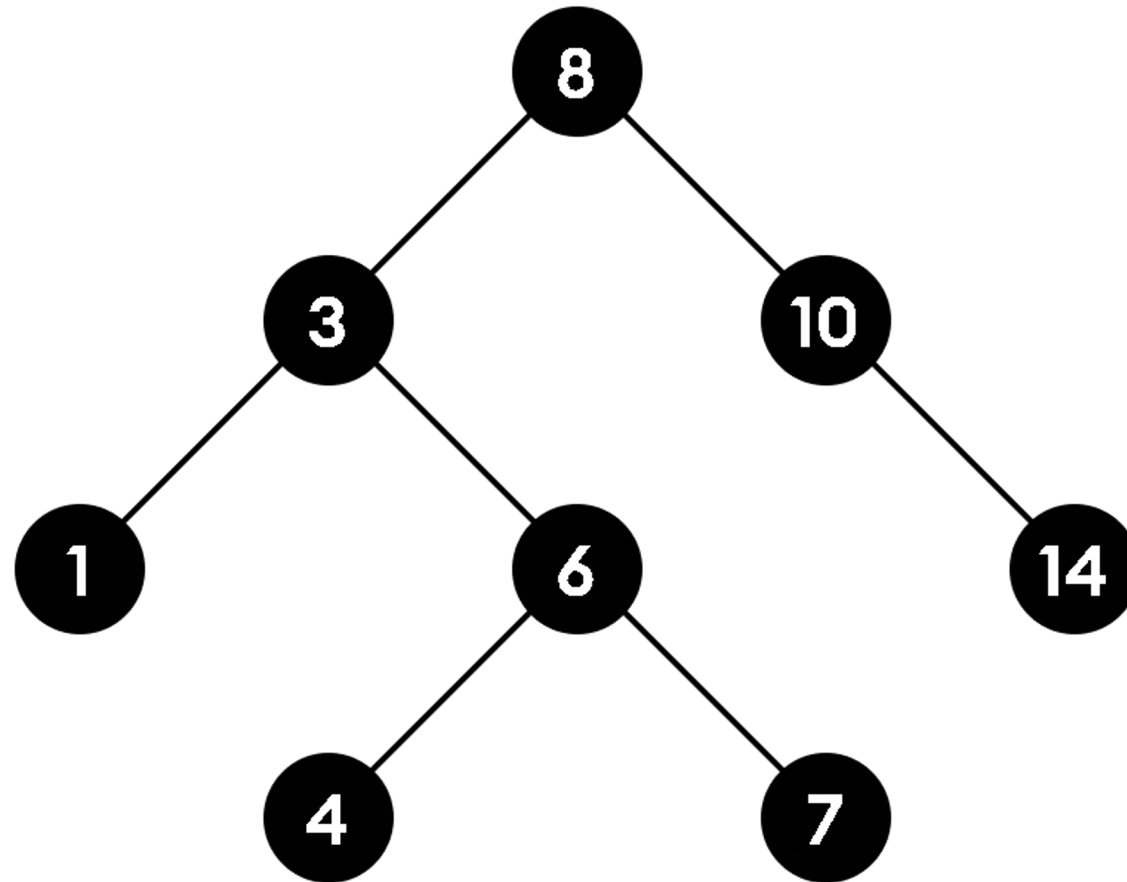
- Get the inorder successor of that node.
- Replace the node with the inorder successor.
- Remove the inorder successor from its original position.

Deletion Operation



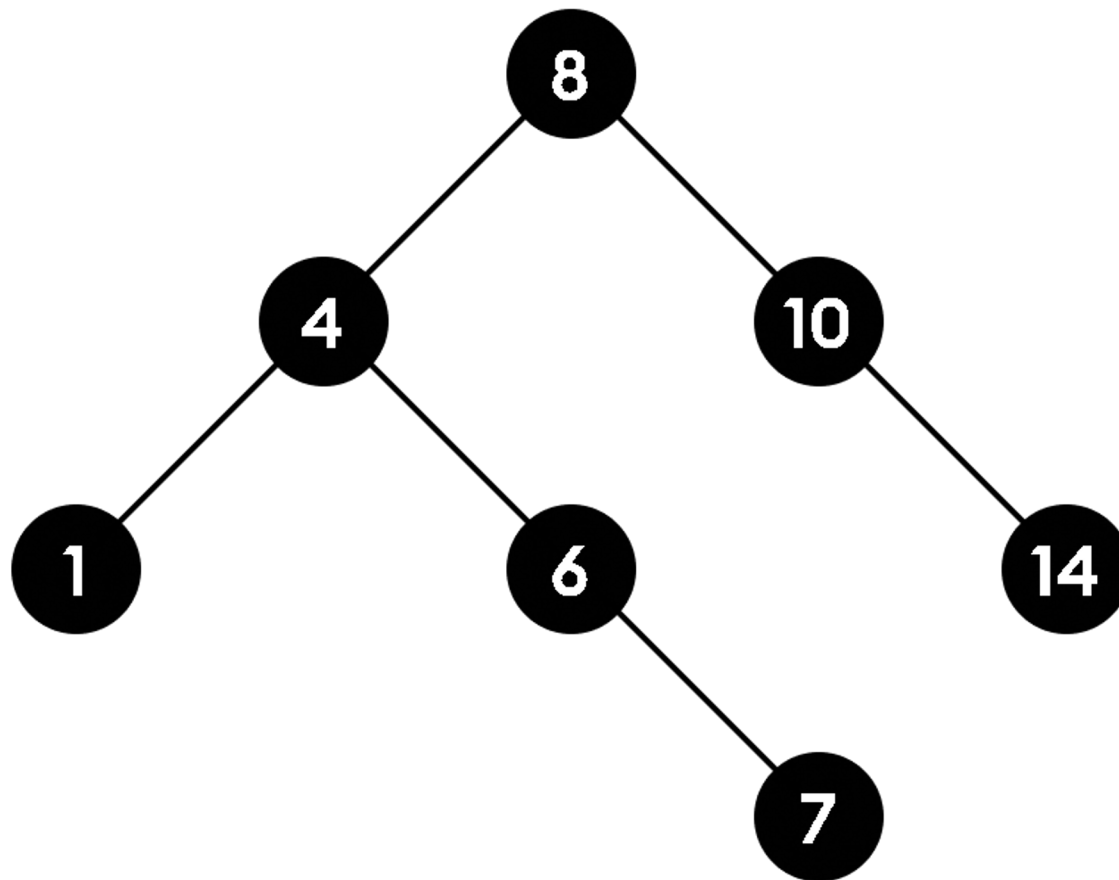
3 to be deleted

Deletion Operation



Copy the value of the inorder successor (4) to the node

Deletion Operation



Delete the inorder successor

Deletion Operation

```
void delete(int data) {  
    root = deleteNode(root, data);  
}  
  
Node deleteNode(Node root, int data) {  
    // Return if the tree is empty  
    if (root == null)  
        return root;  
    // Find the node to be deleted  
    if (data < root.data)  
        root.left = deleteNode(root.left, data);  
    else if (data > root.data)  
        root.right = deleteNode(root.right, data);  
}
```

Deletion Operation

```
else {  
    // If the node is with only one child or no child  
    if (root.left == null)  
        return root.right;  
    else if (root.right == null)  
        return root.left;  
    // If the node has two children  
    // Place the inorder successor in position of the  
    // node to be deleted  
    root.data = in_successor(root.right);  
    // Delete the inorder successor  
    root.right = deleteNode(root.right, root.data);  
}    return root; }
```

Deletion Operation

```
// Find the inorder successor  
int in_successor(Node root) {  
    int val = root.data;  
    while (root.left != null) {  
        val = root.left.data;  
        root = root.left;  
    }  
    return val;  
}
```

AVL Tree

AVL tree is a self-balancing binary search tree in which each node maintains extra information called a balance factor whose value is either -1 , 0 or $+1$.

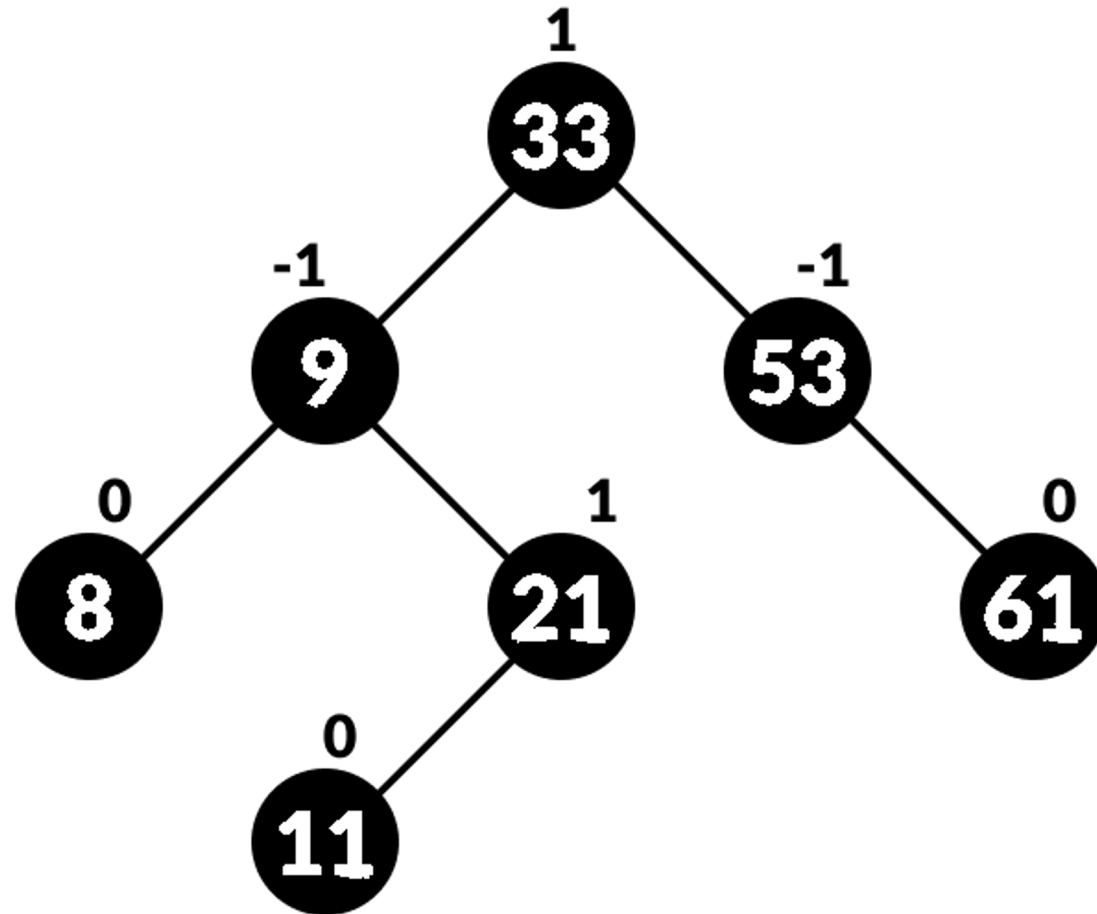
Balance Factor

Balance factor of a node in an AVL tree is the difference between the height of the left subtree and that of the right subtree of that node

Balance Factor = (Height of Left Subtree – Height of Right Subtree) or (Height of Right Subtree – Height of Left Subtree)

The self balancing property of an avl tree is maintained by the balance factor. The value of balance factor should always be -1, 0 or +1.

Balanced avl tree



Operations on an AVL tree

Rotating the subtrees in an AVL Tree :

There are two types of rotations:

Left Rotate:

In left-rotation, the arrangement of the nodes on the right is transformed into the arrangements on the left node.

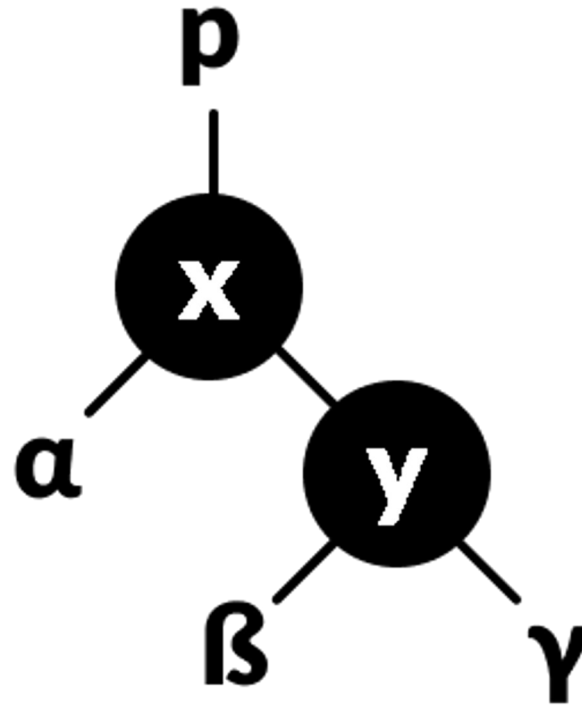
Right Rotate:

In left-rotation, the arrangement of the nodes on the left is transformed into the arrangements on the right node.

Operations on an AVL tree

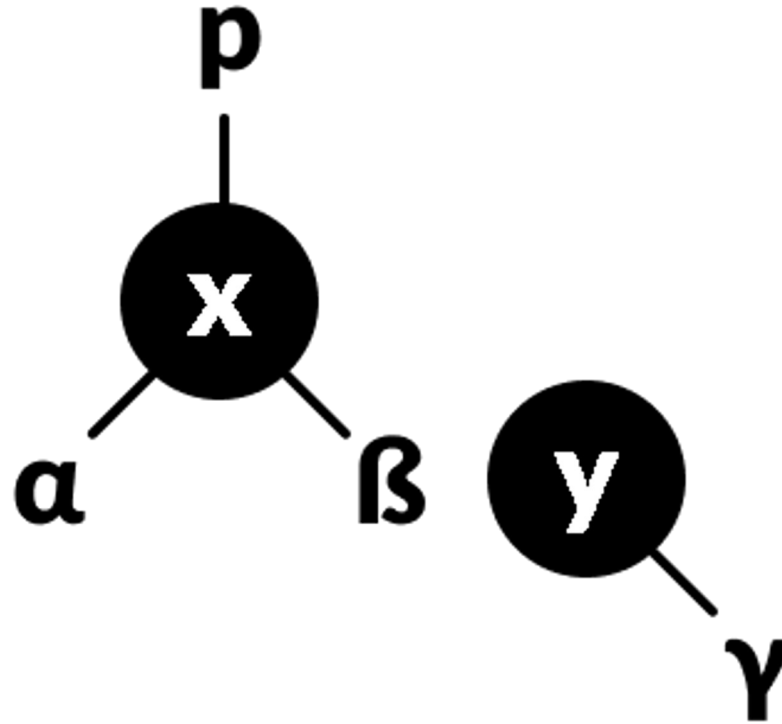
Left Rotate: Algorithm

Let the initial tree be:



Operations on an AVL tree

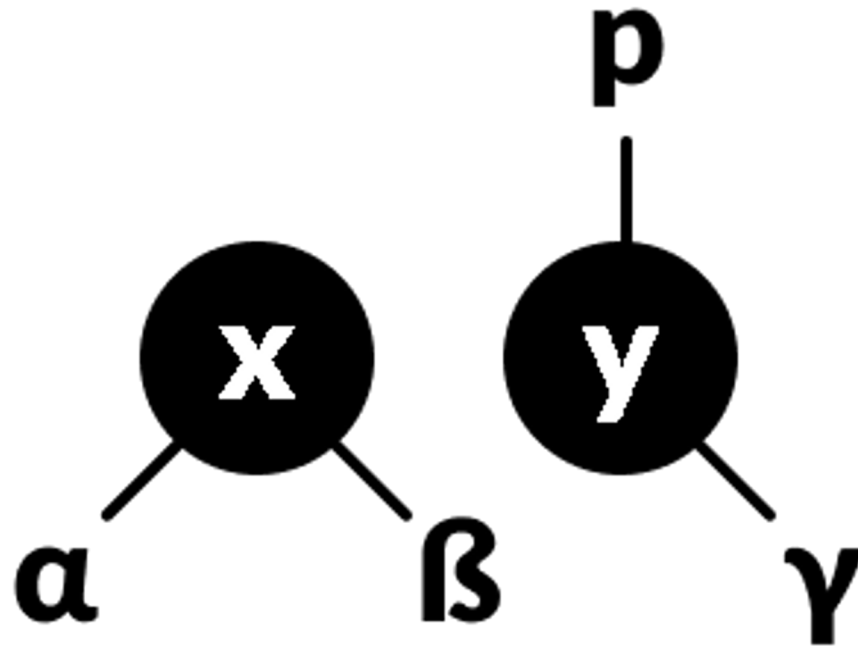
If y has a left subtree, assign x as the parent of the left subtree of y .



Assign x as the parent of the left subtree of y

Operations on an AVL tree

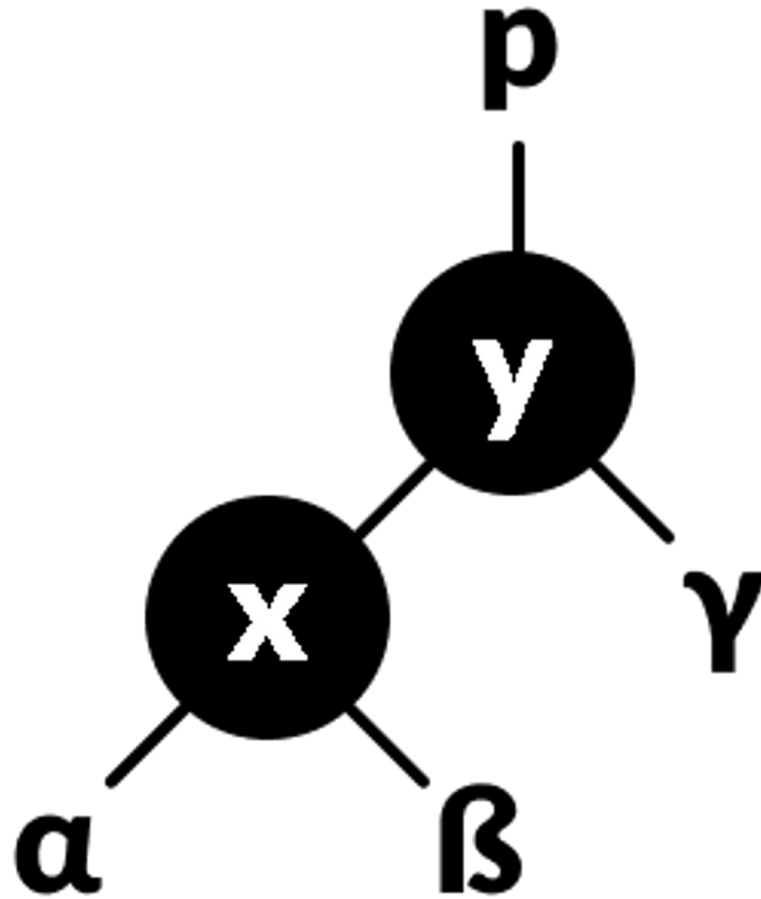
- If the parent of x is NULL, make y as the root of the tree.
- Else if x is the left child of p , make y as the left child of p .
- Else assign y as the right child of p .



Change the parent of x to that of y

Operations on an AVL tree

- Make y as the parent of x .

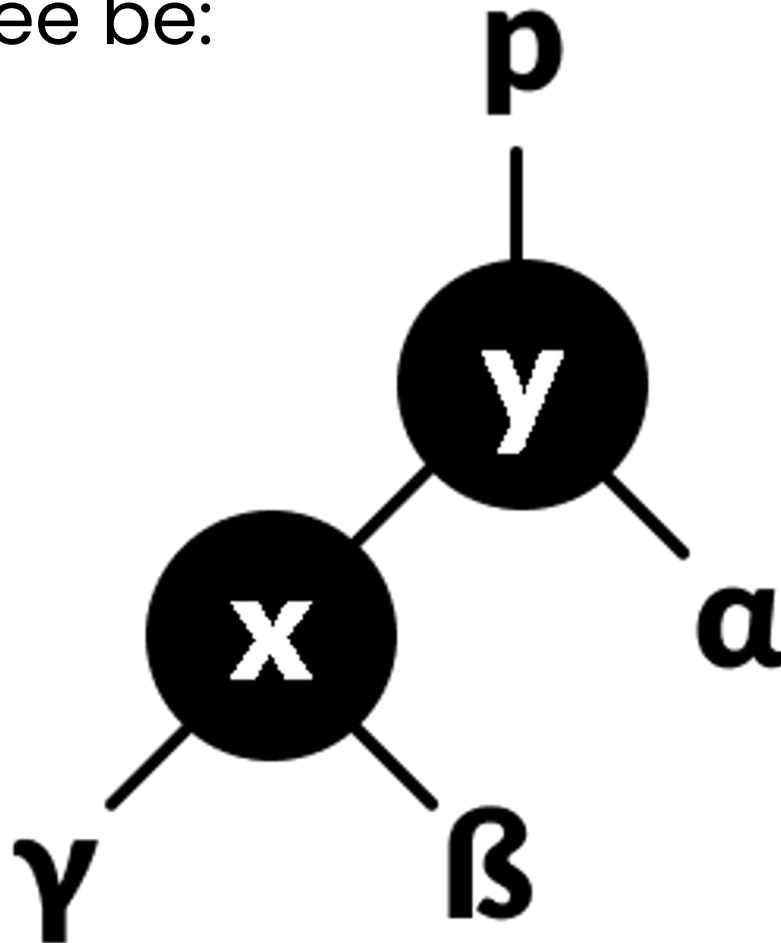


Assign y as the parent of x .

Operations on an AVL tree

Right Rotate : Algorithm

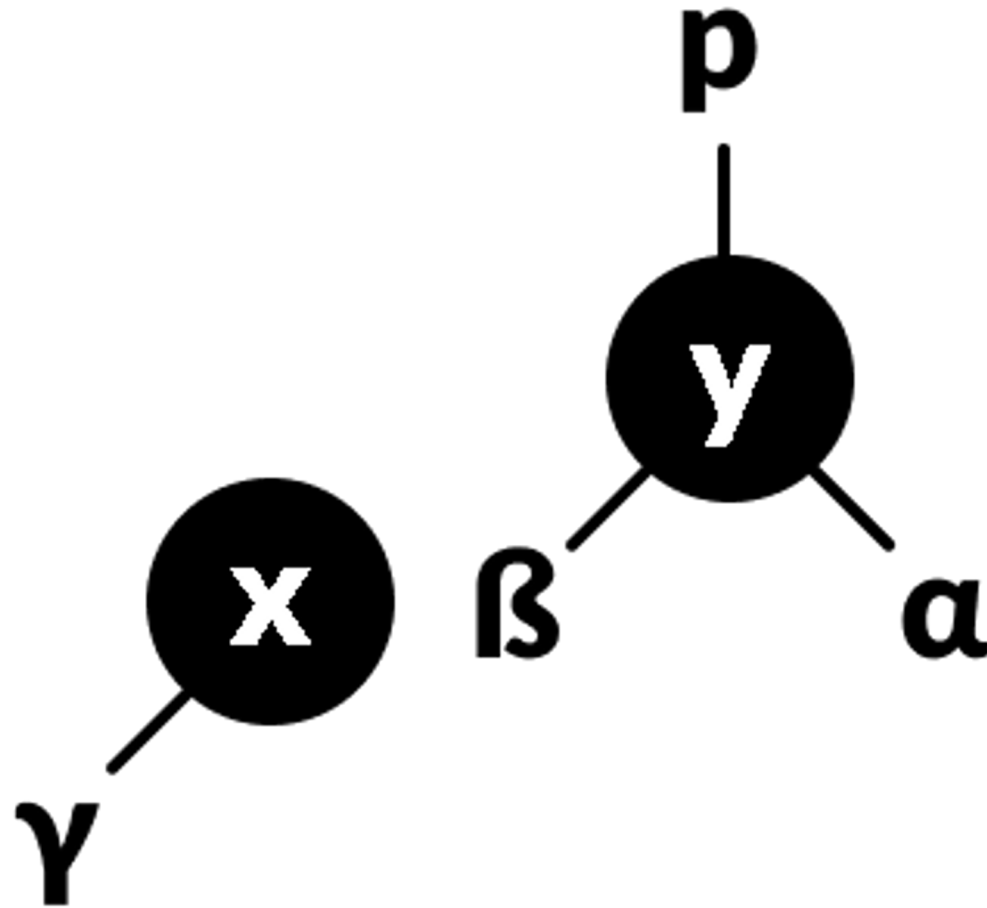
Let the initial tree be:



Assign y as the parent of x.

Operations on an AVL tree

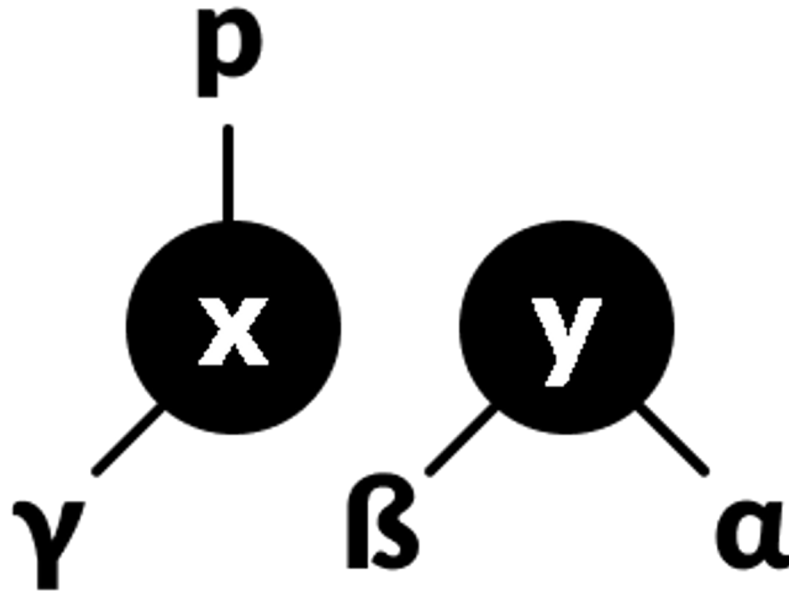
- If x has a right subtree, assign y as the parent of the right subtree of x .



Assign y as the parent of the right subtree of x

Operations on an AVL tree

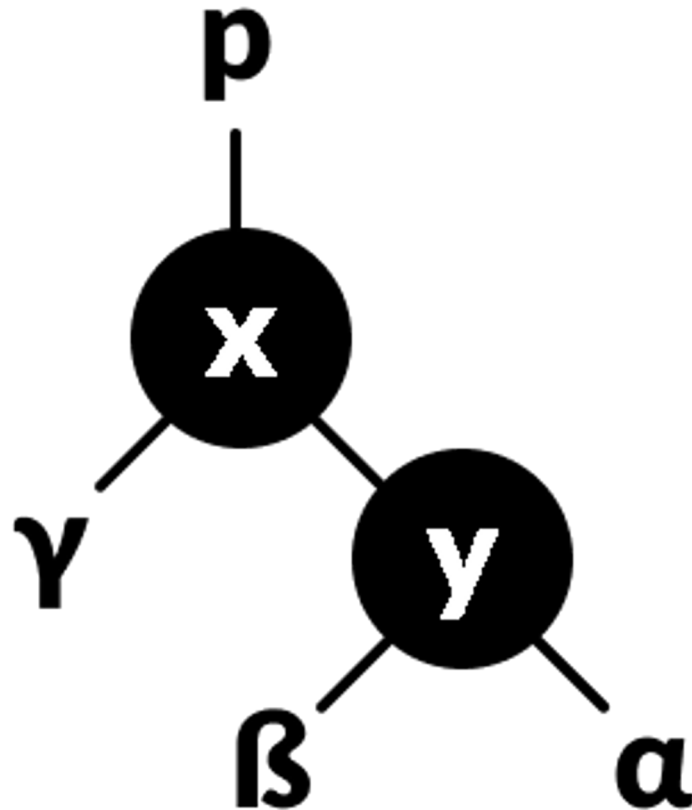
- If the parent of y is NULL, make x as the root of the tree.
- Else if y is the right child of its parent p , make x as the right child of p .
- Else assign x as the left child of p .



Assign the parent of y as the parent of x .

Operations on an AVL tree

- Make x as the parent of y .



Assign x as the parent of y

Implementation:

```
class Node {  
    int item, height;  
    Node left, right;
```

```
Node(int d) {  
    item = d;  
    height = 1;  
}  
}
```

Implementation:

```
//Tree class
class AVLTree {
Node root;
int height(Node N) {
if (N == null)
return 0;
return N.height;
}
int max(int a, int b) {
return (a > b) ? a : b;
}
```

Implementation:

```
Node rightRotate(Node y) {  
    Node x = y.left;  
    Node T2 = x.right;  
    x.right = y;  
    y.left = T2;  
    y.height = max(height(y.left),  
                    height(y.right)) + 1;  
    x.height = max(height(x.left),  
                   height(x.right)) + 1;  
    return x;  
}
```

Implementation:

```
Node leftRotate(Node x) {  
    Node y = x.right;  
    Node T2 = y.left;  
    y.left = x;  
    x.right = T2;  
    x.height = max(height(x.left),  
                    height(x.right)) + 1;  
    y.height = max(height(y.left),  
                   height(y.right)) + 1;  
    return y;  
}
```

Implementation:

```
// Get balance factor of a node
int getBalanceFactor(Node N) {
    if (N == null)
        return 0;
    return height(N.left) - height(N.right);
}
```

Implementation:

```
// Insert a node
Node insertNode(Node node, int item) {
// Find the position and insert the node
if (node == null)
return (new Node(item));
if (item < node.item)
node.left = insertNode(node.left, item);
else if (item > node.item)
node.right = insertNode(node.right, item);
else
return node;
```

Implementation:

```
// Update the balance factor of each node
// And, balance the tree
node.height = 1 + max(height(node.left),
height(node.right));
int balanceFactor = getBalanceFactor(node);
if (balanceFactor > 1) {
if (item < node.left.item) {
return rightRotate(node);
} else if (item > node.left.item) {
node.left = leftRotate(node.left);
return rightRotate(node);
}
}
```

Implementation:

```
if (balanceFactor < -1) {  
    if (item > node.right.item) {  
        return leftRotate(node);  
    } else if (item < node.right.item) {  
        node.right = rightRotate(node.right);  
        return leftRotate(node);  
    }  
}  
return node;  
}
```

Implementation:

```
Node nodeWithMimumValue(Node node) {  
    Node current = node;  
    while (current.left != null)  
        current = current.left;  
    return current;  
}
```

Implementation:

```
// Delete a node
Node deleteNode(Node root, int item) {
    // Find the node to be deleted and remove it
    if (root == null)
        return root;
    if (item < root.item)
        root.left = deleteNode(root.left, item);
    else if (item > root.item)
        root.right = deleteNode(root.right, item);
    else {
        if ((root.left == null) || (root.right ==
            null)) {
            Node temp = null;
```

Implementation:

```
if (temp == root.left)
temp = root.right;
else
temp = root.left;
if (temp == null) {
temp = root; root = null;
} else
root = temp;
} else {
Node temp = nodeWithMimumValue(root.right);
root.item = temp.item;
root.right = deleteNode(root.right, temp.item);
} }
```

Implementation:

```
if (root == null)
return root;
// Update the balance factor of each node and
balance the tree
root.height = max(height(root.left),
height(root.right)) + 1;
int balanceFactor = getBalanceFactor(root);
if (balanceFactor > 1) {
if (getBalanceFactor(root.left) >= 0) {
return rightRotate(root);
} else {
root.left = leftRotate(root.left);
return rightRotate(root); } }
```

Implementation:

```
if (balanceFactor < -1) {  
    if (getBalanceFactor(root.right) <= 0) {  
        return leftRotate(root);  
    } else {  
        root.right = rightRotate(root.right);  
        return leftRotate(root);  
    }  
}  
return root;  
}
```

Implementation:

```
void preorder(Node node) {  
    if (node != null) {  
        System.out.print(node.item + " ");  
        preorder(node.left);  
        preorder(node.right);  
    }  
}
```

Implementation:

```
// Print the tree
private void printTree(Node currPtr, String
indent, boolean last) {
if (currPtr != null) {
System.out.print(indent);
if (last) {
System.out.print("R----");      indent += " ";
} else {
System.out.print("L----");      indent += "| ";
}
System.out.println(currPtr.item);
printTree(currPtr.left, indent, false);
printTree(currPtr.right, indent, true);    }    }
```

Implementation:

```
// Driver code
public static void main(String[] args) {
    AVLTree tree = new AVLTree();
    tree.root = tree.insertNode(tree.root, 33);
    tree.root = tree.insertNode(tree.root, 13);
    tree.root = tree.insertNode(tree.root, 53);
    tree.root = tree.insertNode(tree.root, 9);
    tree.printTree(tree.root, "", true);
    tree.root = tree.deleteNode(tree.root, 13);
    System.out.println("After Deletion: ");
    tree.printTree(tree.root, "", true);
}
}
```

Dynamic Programming

Dynamic Programming is a technique in computer programming that helps to efficiently solve a class of problems that have overlapping subproblems and optimal substructure property.

Dynamic Programming Example

Let's find the fibonacci sequence upto 5th term

Algorithm :

Let n be the number of terms.

- If $n \leq 1$, return 1.
- Else, return the sum of two preceding numbers.

How Dynamic Programming Works

Dynamic programming works by storing the result of subproblems so that when their solutions are required, they are at hand and we do not need to recalculate them.

This technique of storing the value of subproblems is called memoization. By saving the values in the array, we save time for computations of sub-problems.